

$3 \times n$ Grids

Lillian Hart

Last Updated: 7/13/26

1 Preliminaries

For all $3 \times n$ grids, I will be using the following formulation.

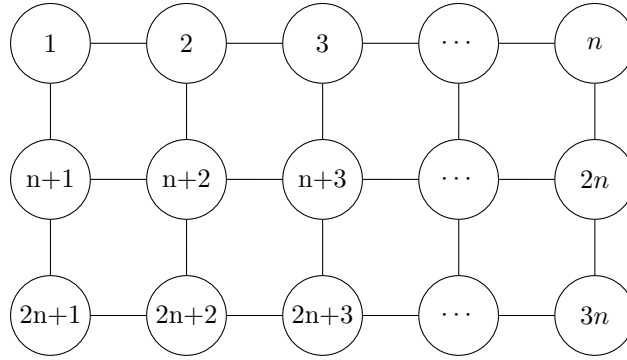


Figure 1: $3 \times n$ Grid

First, I will begin by defining sequences of local complementations and vertex deletions to define protocols. These protocols will be useful when proving border cases.

Definition 1.1. Let $G = (V, N)$ be a $3 \times n$ grid. Suppose $n \geq 3$ and let vertex $a = n + x$ for some $x \in \mathbb{N}$ s.t $1 < x < n$, see figure 2. Define the **Crossing Protocol** to be

$$\chi_a(G) = [\tau_x \circ \tau_a \circ \tau_x(G)] \setminus a \setminus x \setminus (2n + x) \quad (1)$$

If $n \geq 3$, it can be shown that the resulting graph will look like figure 2, if we choose $a = n + x$ s.t $1 < x < n$. See appendix A

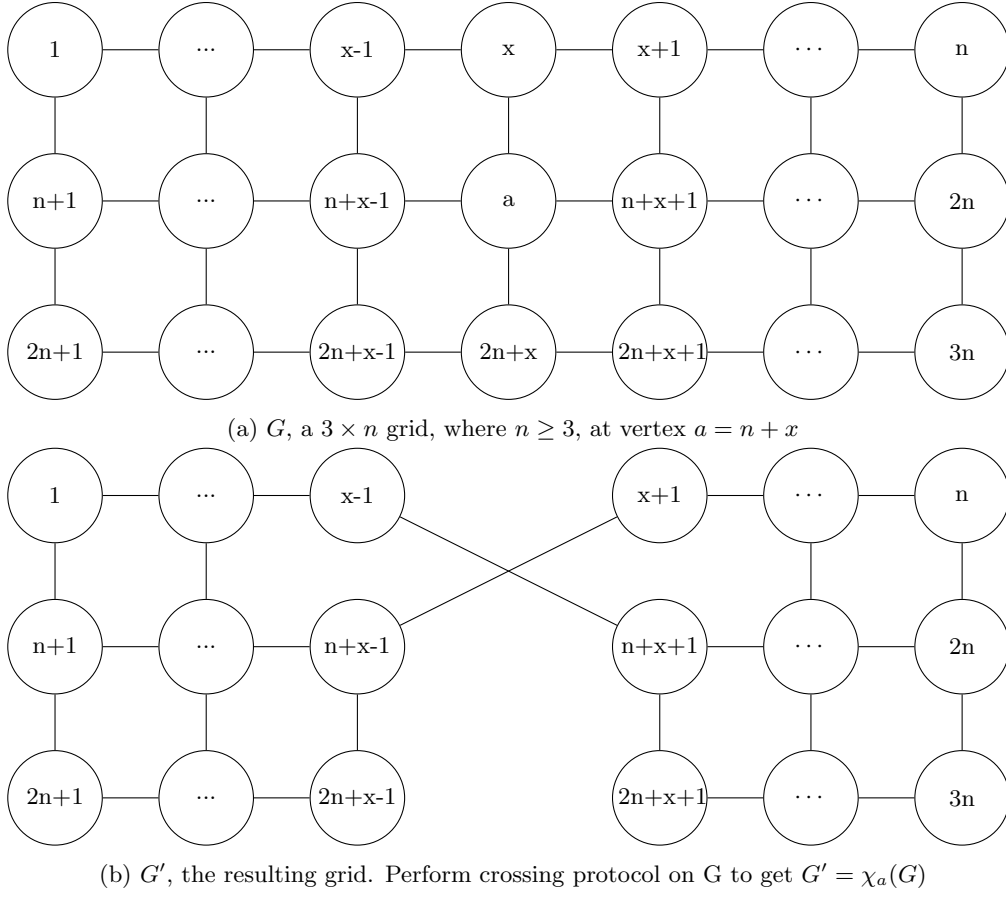


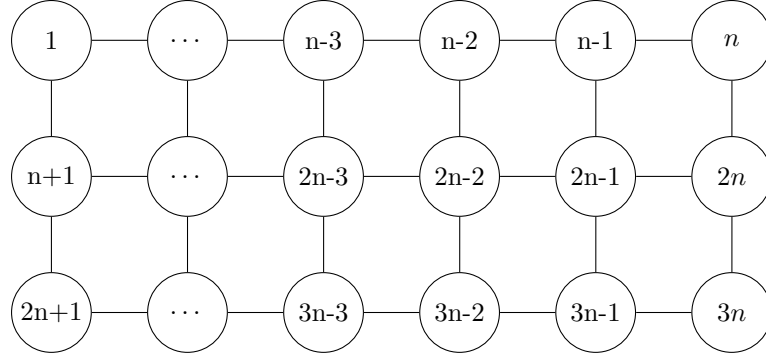
Figure 2

Definition 1.2. Let $G = (V, N)$ be a $3 \times n$ grid. Suppose $n \geq 4$. Define the **Flipping Protocol** to be

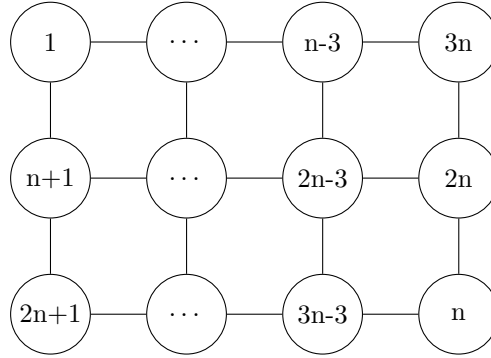
$$\varphi(G) = Y_{3n-2} \circ Y_{2n-2} \circ Y_{n-2} \circ \tau_{2n} \circ \tau_{3n} \circ \tau_{2n} \circ Z_{3n-1} \circ X_{2n-1}^{3n-1} \circ X_{n-1}^n(G) \quad (2)$$

If $n \geq 4$, $G' < G$ where G' is a $3 \times (n-2)$ grid. It can be shown that the resulting graph, G' , will look like G , with the $n-1$ column deleted and the n column replacing the $n-2$ column. Additionally, the corner vertices, $\{n\}$ and $\{3n\}$ will have swapped places. See fig 3. See appendix B.

Definition 1.3. Let $G = (V, N)$ be a $3 \times n$ grid, where $n \geq 3$. Define the **Corner Extraction Protocol** to be
if $n = 3$



(a) G , a $3 \times n$ grid, where $n \geq 3$



(b) G' , the resulting grid. Perform flipping protocol on G to get $G' = \varphi(G)$

Figure 3

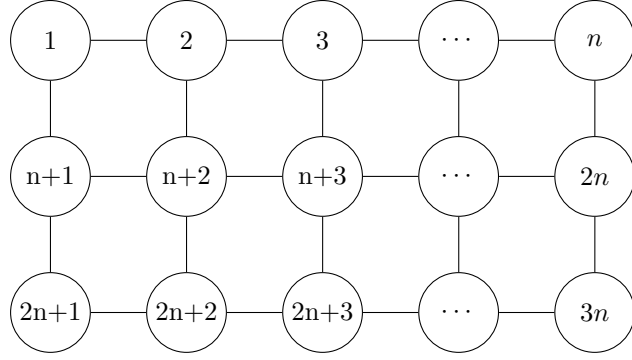
$$\kappa(G) = Y_9 \circ Y_6 \circ Z_8 \circ X_5^8 \circ Y_2(G) \quad (3)$$

if $n \geq 4$

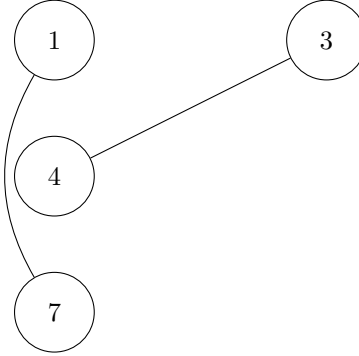
$$\kappa(G) = Z_{2n+4} \circ Z_{n+4} \circ Y_{2n+3} \circ Y_{n+3} \circ Z_{2n+2} \circ X_{n+2}^{2n+2} \circ Y_2(G) \quad (4)$$

If $n = 3$, $G' < G$ where G' contains the extracted bell pairs $\{1, 7\}$ and $\{4, 3\}$. If $n \geq 4$, $G' < G$ where G' is a $3 \times (n - 4)$ grid containing all remaining columns greater than 4. Additionally, the vertices $\{1, 2n + 1\}$ have been extracted and the vertices $\{n + 1, 3, 4\}$ form a line graph attached to the top left corner of the $3 \times (n - 4)$ grid. See fig 4. See appendix C.

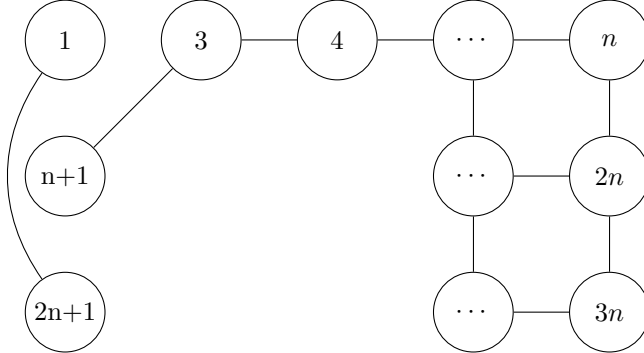
Definition 1.4. Let $G = (V, N)$ be a $2 \times n$ grid, where $n \geq 3$. Let vertex c be the corner vertex, $c = n + 1$, and vertex t s.t $c < t < 2n$. Define the **Corner Replacement Protocol** to be



(a) G , a $3 \times n$ grid, where $n \geq 3$.



(b) For $n = 3$: G' , the resulting grid. Perform the corner extraction protocol on G to get G' .



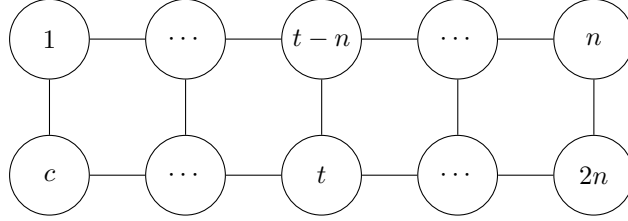
(c) For $n \geq 4$: G' , the resulting grid. Perform the corner extraction protocol on G to get G' .

Figure 4

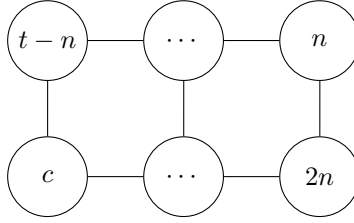
$$\rho_c^t(G) = \tau_c \circ Y_t \circ \cdots \circ Y_{c+2} \circ Y_{c+1} \circ Z_{t-n-1} \circ \cdots \circ Z_2 \circ Z_1(G) \quad (5)$$

It can be shown that $G' < G$ where G' is a $2 \times (n - (t - n) + 1)$ grid with corner vertices $V_c \in \{t - n, c, n, 2n\}$. See fig 5. See appendix E.

*Note: If vertex c is a right corner or a top corner, this protocol can be applied by inverting the vertices the operations are done on horizontally (if right side) or vertically (if top row).



(a) G is a $2 \times n$ grid, where $n \geq 3$. Vertex $c = n + 1$, and vertex t is s.t $c < t < 2n$.



(b) G' , the resulting grid. G' is a $2 \times (n - (t - n) + 1)$ grid with corner vertices $V_c \in \{t - n, c, n, 2n\}$.

Figure 5

Definition 1.5. Let $G = (V, N)$ be a $2 \times n$ grid, where $n \geq 5$. Define the **Corner Folding Protocol** to be

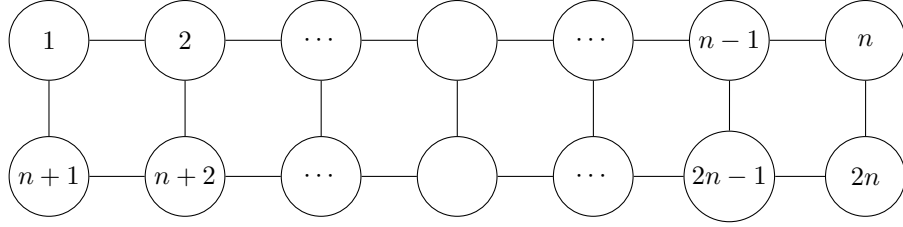
$$\xi(G) = \tau_{2n} \circ \tau_{n+1} \circ Y_{n-1} \circ Y_2 \circ Y_n \circ Y_1 \circ \tau_{n-1} \circ \tau_2 \circ \tau_n \circ \tau_1 \circ \tau_{2n} \circ \tau_{n+1}(G) \quad (6)$$

It can be shown that $G' < G$ where G' is a $2 \times (n - 2)$ grid with corner vertices $V_c \in \{n + 1, n + 2, 2n, 2n - 1\}$. See fig 6. See appendix F.

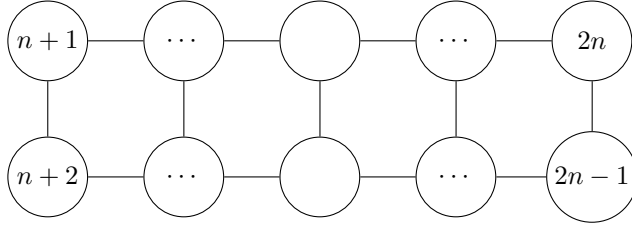
*Note: If you want to 'fold over' the top corners, a similar protocol can be used by vertically inverting the vertices the operations are being performed on.

2 Border Cases: Sides

In this section we look at constructions for $3 \times n$ grids. We focus on the side border cases where the vertices of the bell pairs lie in either the first column, or the last column. We



(a) G is a $2 \times n$ grid, where $n \geq 5$.



(b) G' , the resulting grid. G' is a $2 \times (n - (t - n) + 1)$ grid with corner vertices $V_c \in \{t - n, c, n, 2n\}$.

Figure 6

determined that for all possible arrangements of bell pairs, we can simultaneously extract them.

2.1 Non-Crossing Bell Pairs

Lemma 2.1. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 2$, then it is possible to simultaneously extract Bell pairs $\{1, n\}$ and $\{2n + 1, 3n\}$. See figure 7

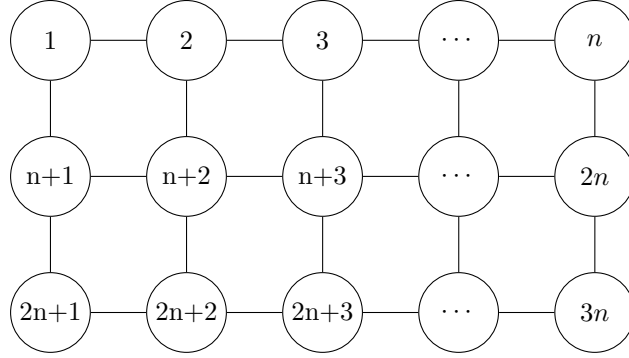
Proof. Let $n \geq 2$, then $G' < G$, where G' consists of two separate line graphs, each containing one bell pair.

First, perform Z measurements on the middle row so that $G' = Z_{2n} \circ \dots \circ Z_{n+1}(G)$. The resulting graph is two separate line graphs with one containing $\{1, n\}$ as end points and the other containing $\{2n + 1, 3n\}$ as end points.

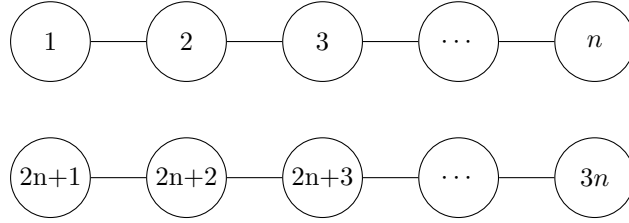
Then simply perform Y measurements on all intermediary vertices, so that $G'' = Y_{n-1} \circ \dots \circ Y_2 \circ Y_{3n-1} \circ \dots \circ Y_{2n+2}(G')$. $\square$

Lemma 2.2. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{1, n\}$ and $\{n + 1, 2n\}$.

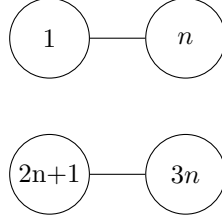
Proof. Let $n \geq 4$, then it is easy to see that $G' < G$, where G' is a $2 \times n$ graph, with corner vertices $V_c \in \{1, n, n + 1, 2n\}$. We can achieve this by performing Z measurements along the 3rd row.



(a) G is a $3 \times n$ grid where $n \geq 2$. We will simultaneously extract $\{1, n\}$ and $\{2n+1, 3n\}$.



(b) Perform Z measurements along all vertices in the middle row.



(c) Perform Y measurements on all vertices except $\{1, n, 2n+1, 3n\}$.

Figure 7

Using lemma 1 from Megan's paper, $\{1, n\}$ and $\{n+1, 2n\}$ are simultaneously extractable. $\square$

Lemma 2.3. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{1, n\}$ and $\{2n+1, 2n\}$. See figure 8

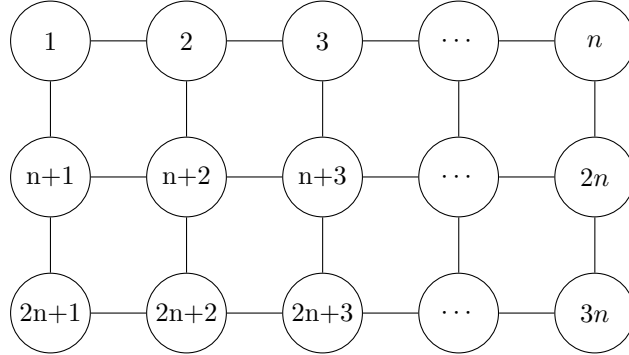
Proof. Let $n \geq 4$, then $G' < G$, where G' is a $2 \times n$ graph, with corner vertices $V_c \in \{1, n, 2n+1, 2n\}$.

Begin by performing vertex deletions on all vertices on the bottom row, except vertex $\{2n+1\}$. Then, $G_1 = Z_{3n} \circ \dots \circ Z_{2n+2}(G)$ looks like a $2 \times n$ grid with the vertex $\{2n+1\}$ being a leaf attached to the corner vertex, $V_c = \{n+1\}$, of the $2 \times n$ grid.

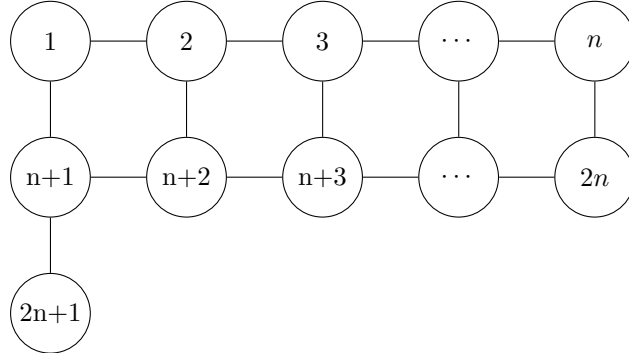
Next, perform a Y measurement on vertex $\{n+1\}$. Notice, that the only neighbors of

$\{n+1\}$ are vertices $\{1, n+2, 2n+1\}$. This results in $G_2 = Y_{n+1}(G_1)$ looking like a $2 \times n$ grid with corner vertices $V_c = \{1, n, 2n+1, 2n\}$ and an additional edge $(1, n+2)$.

Finally, perform a local complementation on $\{2n+1\}$ to get rid of that extra edge. Then $G' = \tau_{2n+1}(G_2)$, which is a $2 \times n$ grid with corner vertices $V_c = \{1, n, 2n+1, 2n\}$. By lemma 1 in Megan's work, Bell Pairs $\{1, n\}$ and $\{2n+1, 2n\}$ are simultaneously extractable for $n \geq 4$. $\square$



(a) G is a $3 \times n$ grid where $n \geq 4$. We will simultaneously extract $\{1, n\}$ and $\{2n+1, 2n\}$.



(b) $G_1 = Z_{3n} \circ \dots \circ Z_{2n+2}(G)$; Perform Z measurements along all vertices in the bottom row except $\{2n+1\}$.

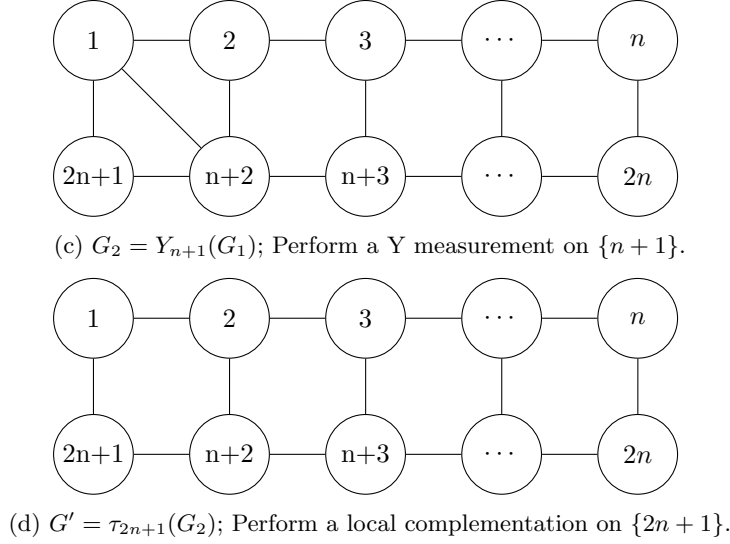


Figure 8

Lemma 2.4. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{1, n\}$ and $\{n+1, 2n+1\}$. See figure 9

Proof. Let $n \geq 4$, then $G' < G$, where G' consists of two separate line graphs, each containing one bell pair.

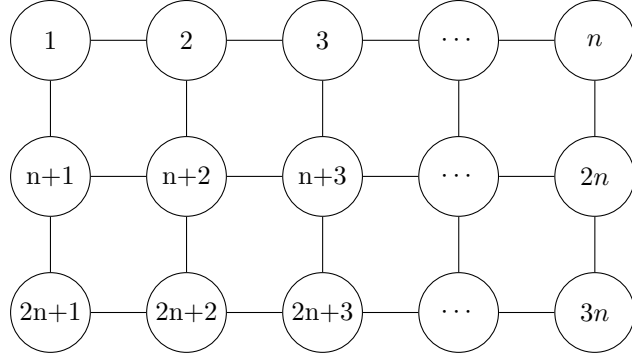
Begin by performing the crossing protocol on vertex $\{n+2\}$ so that $G_1 = \chi_{n+2}(G)$. The resulting graph has vertex $\{1\}$ connected to vertex $\{n+3\}$ as a leaf which is then connected to the greater grid. Additionally, vertex $\{n+1\}$ is connected to vertex $\{3\}$ as a leaf which is then connected to the greater grid. It is also important to note that vertex $\{2n+1\}$ is attached to $\{n+1\}$ as a leaf.

Next, perform a Z measurement on vertex $\{3\}$. Notice, that we have now extracted the $\{n+1, 2n+1\}$ bell pair. This results in $G_2 = Z_3(G_1)$ containing one extracted bell pair.

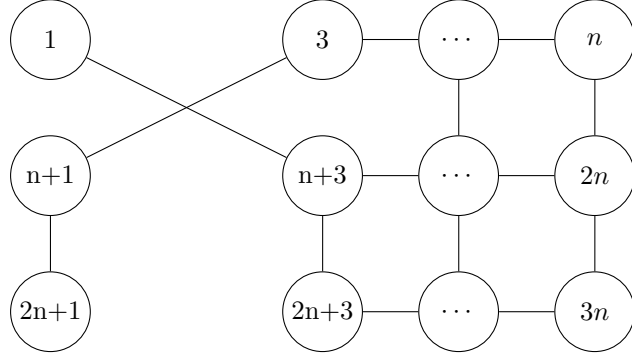
It is not hard to see that by choosing any path between vertices $\{1, n\}$ and performing Z measurements on the vertices not in that path, you can extract a line graph with $\{1, n\}$ as the end points, G' .

Finally, by performing Y measurement on all vertices on the path between $\{1, n\}$, you can extract the second bell pair. Thus Bell Pairs $\{1, n\}$ and $\{2n+1, 2n\}$ are simultaneously extractable for $n \geq 4$. $\square$

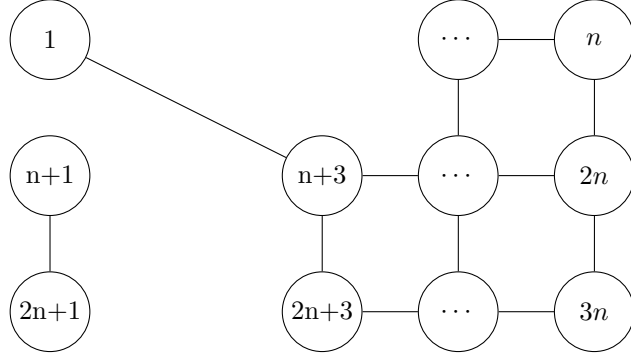
*Note: this solution also works for simultaneously extracting $\{1, 2n\}$ and $\{n+1, 2n+1\}$, and $\{1, 3n\}$ and $\{n+1, 2n+1\}$, for $n \geq 3$.



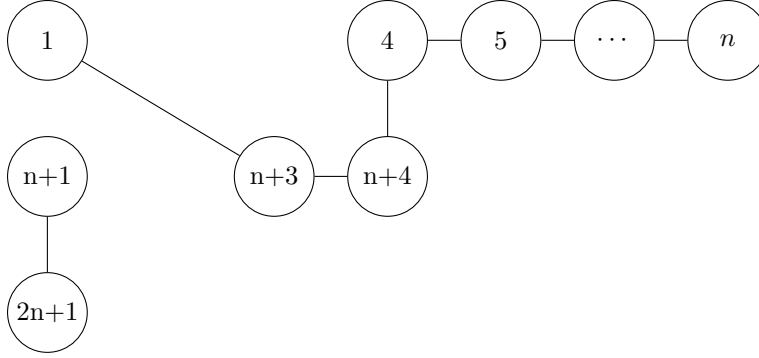
(a) G is a $3 \times n$ grid where $n \geq 4$. We will simultaneously extract $\{1, n\}$ and $\{n+1, 2n+1\}$.



(b) $G_1 = \chi_{n+2}(G)$; Perform the crossing protocol on $\{n+2\}$. See fig 2



(c) $G_2 = Z_3(G_1)$; Perform a Z measurement on vertex $\{3\}$.



(d) G' ; Choose a path between $\{1, n\}$ and perform Z measurements on remaining nodes in the grid that are not on the path.

Figure 9

We believe that for certain cases these lemmas could be extended to an $m \times n$ grid; However, more work will need to be done to determine cases where these protocols could not be used in the extended grid.

2.2 Crossing Bell Pairs

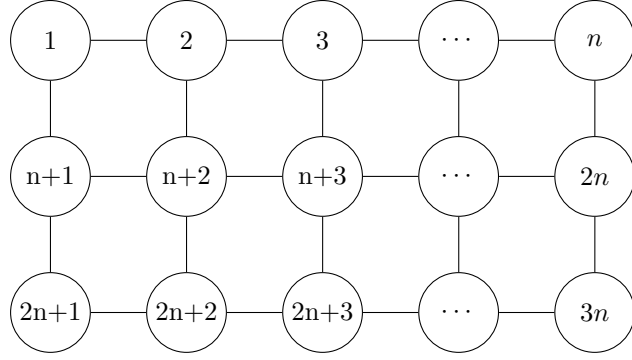
Lemma 2.5. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 3$, then it is possible to simultaneously extract Bell pairs $\{1, 3n\}$ and $\{n, 2n + 1\}$. See figure 10

Proof. Let $n \geq 3$, then $G' < G$, where G' consists of two separate line graphs, each containing one bell pair.

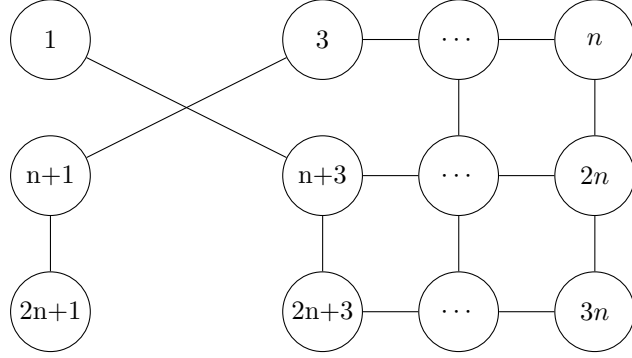
Begin by performing the crossing protocol on vertex $\{n + 2\}$ so that $G_1 = \chi_{n+2}(G)$. The resulting graph has vertex $\{1\}$ connected to vertex $\{n + 3\}$ as a leaf which is then connected to the greater grid. Additionally, vertex $\{n + 1\}$ is connected to vertex $\{3\}$ as a leaf which is then connected to the greater grid. It is also important to note that vertex $\{2n + 1\}$ is attached to $\{n + 1\}$ as a leaf. Finally, as a result of the crossing protocol, there is no edge between $(3, n + 3)$

It is not hard to see that by performing Z measurements on all the vertices in the middle row, right of vertex $\{n + 3\}$, you can extract two line graphs with $\{1, 3n\}$ and $\{n, 2n + 1\}$ as the end points, $G' = Z_{2n} \circ \dots \circ Z_{n+4}(G_1)$.

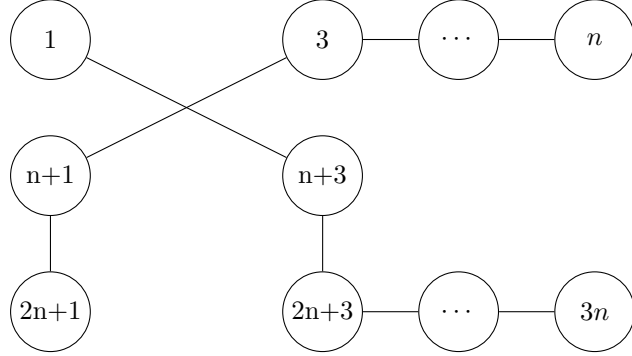
Finally, by performing Y measurement on all vertices on the path between $\{1, 3n\}$ and $\{n, 2n + 1\}$, you can simultaneously extract $\{1, 3n\}$ and $\{n, 2n + 1\}$. $\square$



(a) G is a $3 \times n$ grid where $n \geq 3$. We will simultaneously extract $\{1, 3n\}$ and $\{n, 2n+1\}$.



(b) $G_1 = \chi_{n+2}(G)$; Perform the crossing protocol on $\{n+2\}$. See fig 2



(c) $G' = Z_{2n} \circ \dots \circ Z_{n+4}(G_1)$; Perform Z measurements on middle row vertices, to the right of vertex $\{n+3\}$.

Figure 10

Lemma 2.6. Let $G = (V, E)$ be a $3 \times n$ grid. If $n = 3$ or $n \geq 5$, then it is possible to simultaneously extract Bell pairs $\{1, 2n\}$ and $\{n+1, n\}$.

Proof. Let $n = 3$ or $n \geq 5$, then it is easy to see that $G' < G$, where G' is a $2 \times n$ graph, with

corner vertices $V_c \in \{1, n, n+1, 2n\}$. We can achieve this by performing Z measurements along the 3rd row.

Using lemma 2 from Megan's paper, $\{1, 2n\}$ and $\{n+1, n\}$ are simultaneously extractable. $\square$

Lemma 2.7. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 3$, then it is possible to simultaneously extract Bell pairs $\{1, 2n\}$ and $\{2n+1, n\}$.

Proof. Let $n \geq 3$.

If the graph is a 3×4 grid, before proceeding turn the graph into a 3×3 grid that preserves bell pair vertices. This can be done by deleting vertex $\{n+1\}$. Then perform Y measurements on vertices $\{2, 2n+2\}$. Finally perform local complementations on $\{1, 2n+1\}$. So $G_1 = \tau_{2n+1} \circ \tau_1 \circ Y_{2n+2} \circ Y_1 \circ Z_{n+1}(G)$ and G_1 is a 3×3 grid that preserves all the positions of the bell pair vertices.

Now all the $3 \times n$ grids have $n = 3$ or $n \geq 5$ columns, then $G' < G$, where G' is a $2 \times n$ graph, with corner vertices $V_c \in \{1, n, 2n+1, 2n\}$.

By performing the same sequence of local complementations and vertex deletions as in lemma 2.3, we can achieve the graph G' .

Then G' is a $2 \times n$ grid with corner vertices $V_c = \{1, n, 2n+1, 2n\}$. By lemma 2 in Megan's work, Bell Pairs $\{1, 2n\}$ and $\{2n+1, n\}$ are simultaneously extractable for $n \geq 3$. $\square$

*Note: using a similar technique described here, we can see that for $n \geq 3$ you can also simultaneously extract: $\{1, 3n\}$ and $\{2n+1, 2n\}$; $\{1, 3n\}$ and $\{n+1, n\}$; $\{n+1, 3n\}$ and $\{2n+1, n\}$

Lemma 2.8. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 6$, then it is possible to simultaneously extract Bell pairs $\{1, 3n\}$ and $\{n+1, 2n\}$. See figure 11

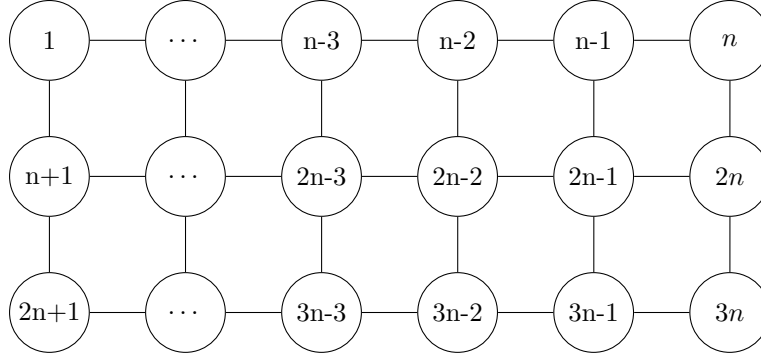
Proof. Let $n \geq 6$, then $G' < G$, where G' is a $3 \times (n-2)$ graph. This graph has the corner vertices $\{n, 3n\}$ flipped and columns $n-1$ and $n-2$ deleted. Finally the column n is shifted over and joined to the rest of the grid.

By performing the flipping protocol as defined in def1.2, we can achieve the graph G' .

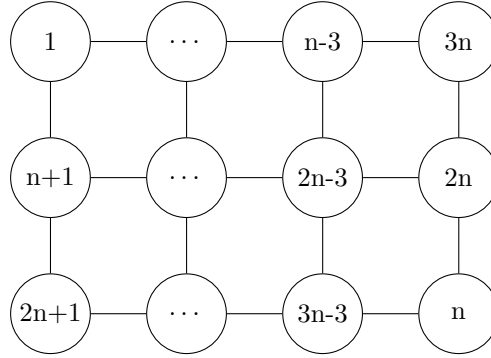
Thus by lemma 2.2, since G' is $3 \times n-2$ grid where $n-2 \geq 4$, the bell pairs $\{1, 3n\}$ and $\{n+1, 2n\}$ are simultaneously extractable. $\square$

*Note: using a similar technique, if $n \geq 6$, then it is possible to simultaneously extract $\{n, 2n+1\}$ and $\{n+1, 2n\}$.

**Note: It is still not certain whether simultaneous extraction of $\{1, 3n\}$ and $\{n+1, 2n\}$ is possible for the $n = 3$, $n = 4$, and $n = 5$ cases.



(a) G is a $3 \times n$ grid where $n \geq 6$. We will simultaneously extract $\{1, 3n\}$ and $\{n+1, 2n\}$.



(b) $G' = \varphi(G)$; Perform the flipping protocol. Then by lemma 2.2 bell pairs $\{1, 3n\}$ and $\{n+1, 2n\}$ are simultaneously extractable.

Figure 11

Lemma 2.9. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 3$, then it is possible to simultaneously extract Bell pairs $\{1, 2n+1\}$ and $\{n+1, n\}$.

Proof. Let $n \geq 3$, then $G' < G$.

Case 1: $n = 3$

If $n = 3$, then perform the corner extraction protocol, see def 1.3. Then G' contains the two extracted bell pairs $\{1, 7\}$ and $\{4, 3\}$. See figure 4b

Case 2: $n = 4$

If $n = 4$, then perform the corner extraction protocol, see def 1.3. Then G' contains the extracted bell pair $\{1, 9\}$. Additionally, it contains a line graph with $\{5, 4\}$ as the endpoints. By performing a Y measurement on vertex $\{3\}$, the second bell pair can be extracted. See figure 4c.

Case 3: $n > 4$

If $n > 4$, then perform the corner extraction protocol, see def 1.3. Then G' contains

the extracted bell pair $\{1, 2n + 1\}$. Additionally, it contains the line graph with vertices $\{n + 1, 3, 4\}$ in that respective order. The line graph is connected to a $2 \times n - 4$ grid via the edge $(4, 5)$. Finally, the $2 \times n - 4$ grid also contains the vertex n . See figure 4c.

Select a path between $\{n + 1, n\}$ and then delete all remaining vertices in the grid. Then perform Y measurements along the path to extract the bell pair $\{n + 1, n\}$.

Thus it is possible to simultaneously extract Bell pairs $\{1, 2n + 1\}$ and $\{n + 1, n\}$

□

*Note: using a similar technique, if $n \geq 3$, then it is possible to simultaneously extract $\{1, 2n + 1\}$ and $\{n + 1, 3n\}$

Lemma 2.10. Let $G = (V, E)$ be a $3 \times n$ grid. If $n \geq 5$, then it is possible to simultaneously extract Bell pairs $\{1, 2n + 1\}$ and $\{n + 1, 2n\}$.

Proof. Let $n \geq 5$, then $G' < G$, where G' contains the extracted bell pair $\{1, 2n + 1\}$. Additionally, it contains the line graph with vertices $\{n + 1, 3, 4\}$ in that respective order. The line graph is connected to a $2 \times n - 4$ grid, which contains vertex $\{2n\}$, via the edge $(4, 5)$. See figure 4c.

By performing the corner extraction protocol, see def 1.3, we can achieve G' .

Select a path between $\{n + 1, 2n\}$ and then delete all remaining vertices in the grid. Then perform Y measurements along the path to extract the bell pair $\{n + 1, 2n\}$.

Thus it is possible to simultaneously extract Bell pairs $\{1, 2n + 1\}$ and $\{n + 1, 2n\}$

□

3 Border Cases: One Row

Here we aim to resolve the line graph bottlenecks presented in [2] when the line graph is extended to grid. We were able to successfully resolve the "nested" bell pair case, and the "adjacent" bell pair case; However, more work will need to be done to determine if the "overlapping" case can be resolved. We suspect that if the line graph is extended to a $3 \times n$ grid, we could implement the crossing protocol to resolve some cases of "overlapping" bell pairs.

Lemma 3.1. Let $G = (V, E)$ be a $2 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$, where $kn + 1 \leq a_1 < b_1 < b_2 < a_2 \leq (k + 1)n$ and $b_2 = b_1 + 1$, for some $0 \leq k \leq 1$. See appendix D

Proof. Let $n \geq 4$, then by induction, we can show that bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

Base Case: $n = 4$. See figure 15

If $n=4$, then $\{a_1, b_1, b_2, a_2\}$ must be next to each other, in that order. WLOG suppose $k=1$, so all the vertices are on the 2nd row.

Then $a_1 = 5$, $b_1 = 6$, $b_2 = 7$, and, $a_2 = 8$. Then $G' < G$ where G' contains the extracted bell pairs $\{5, 8\}$ and $\{6, 7\}$.

We can achieve G' by performing the following sequence of operations.

First perform local complementations on vertices $\{5, 8\}$. Then perform Y measurements on vertices $\{1, 4\}$. The resulting graph contain the extracted bell pair $\{6, 7\}$ and a line graph containing $\{5, 2, 3, 8\}$. By performing Y measurements on vertices $\{2, 3\}$, you can extract the second bell pair. $G' = Y_2 \circ Y_3 \circ Y_1 \circ Y_4 \circ \tau_8 \circ \tau_5(G)$

Inductive Step

Now suppose that lemma 3.1 holds for $n \geq 4$. Consider the $2 \times (n+1)$ grid and WLOG assume $k = 1$. Since $\{b_1, b_2\}$ must be neighbors and $n+1 > 4$, there are 4 cases:

Case 1: See figure 16a. Suppose $a_1 > (n+1) + 1$. Then we know $\{a_2, b_1, b_2\}$ must all be greater than $(n+1) + 1$ as well. We can safely perform Z measurements on the first column. Thus the resulting grid is a $2 \times n$ containing $\{a_1, a_2, b_1, b_2\}$, so by our inductive hypothesis $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

Case 2: See figure 16c. Suppose $a_2 < 2(n+1)$. Then we know $\{a_1, b_1, b_2\}$ must all be less than $2(n+1)$ as well. We can safely perform Z measurements on the last column. Thus the resulting grid is a $2 \times n$ containing $\{a_1, a_2, b_1, b_2\}$, so by our inductive hypothesis $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

Case 3: See figure 16e. Suppose $a_1 = (n+1)+1$, $a_2 = 2(n+1)$ and $b_1 = a_1+1 = (n+1)+2$. Then we know $b_2 = (n+1) + 3$ since b_1 and b_2 must be neighbors. We can see that b_2 belongs to the 3rd column. Since $a_2 = 2(n+1)$, a_2 must belong to the $(n+1)$ column. Then since $n \geq 4$, b_2 and a_2 cannot be neighbors.

We can then delete vertex $\{n+1\}$. Next perform a Y measurement on vertex $\{2(n+1) - 1\}$. Finally by performing a local complementation on vertex $2(n+1)$, the resulting grid is $2 \times n$ grid containing $\{a_1, a_2, b_1, b_2\}$, so by our inductive hypothesis $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

Case 4: See figure 16h. Suppose $a_1 = (n+1)+1$, $a_2 = 2(n+1)$ and $b_1 > a_1+1 = (n+1)+2$. Then we know that b_1 and a_1 cannot be neighbors. We also see that $\{a_2, b_2\}$ must be greater than b_1 , so they also do not neighbor a_1 .

We can then delete vertex $\{1\}$. Next perform a Y measurement on vertex $\{(n+1) + 2\}$. Finally by performing a local complementation on vertex $\{a_1 = (n+1) + 1\}$, the resulting grid is $2 \times n$ grid containing $\{a_1, a_2, b_1, b_2\}$, so by our inductive hypothesis $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

If $k=0$, you can use the same technique except performed on inverted vertices to show that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

□

Lemma 3.2. Let $G = (V, E)$ be a $2 \times n$ grid. If $n \geq 6$, then it is possible to simultaneously extract Bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$, where $kn + 1 \leq a_1 < b_1 < b_2 < a_2 \leq (k + 1)n$ and $b_2 > b_1 + 2$, for some $0 \leq k \leq 1$.

Proof. Let $n \geq 6$. WLOG assume that $\{a_1, a_2, b_1, b_2\}$ all lie on the bottom row, so $k = 1$. Then G contains vertices such that $a_1 < b_1 < b_2 < a_2$ and $b_2 > b_1 + 2$. We can show that $G' < G$ where G' is a $2 \times m$ grid with corner vertices $V_c \in \{a_1, b_1, a_2, b_2\}$ and $m \geq 4$.

To do this we first perform vertex deletions so that the bottom corner vertices are $\{a_1, a_2\}$. This is achieved by performing Z measurements on all vertices in columns left of a_1 and right of a_2 .

Next, we will ensure that $\{a_1, b_1\}$ and $\{b_2, a_2\}$ are neighbors. If they are already neighbors then move onto the next step. If one, or both of them are not, use the corner replacement protocol on a_1 and/or a_2 : $\rho_{a_1}^{b_1-1}$ and/or $\rho_{a_2}^{b_2+1}$. See definition 1.4.

Now we have a grid such that $\{a_1, a_2\}$ are the bottom corner vertices. Additionally, $b_1 = a_1 + 1$ and $b_2 = a_2 - 1$. We can use the corner folding protocol, see def 1.5, to move $\{a_1, a_2\}$ to the top corners, while $\{b_1, b_2\}$ are turned into the bottom corners. We also note that the grid is a $2 \times m$ grid, where $m = b_2 - b_1 + 1$. Since $b_2 > b_1 + 2$, $m > 3$, so $m \geq 4$.

Thus G' is a $2 \times m$ grid with corner vertices $V_c \in \{a_1, b_1, a_2, b_2\}$ and $m \geq 4$. Then by megan's work we can see that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

If $k=0$, you can use the same technique except performed on inverted vertices to show that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable. □

Lemma 3.3. Let $G = (V, E)$ be a $2 \times n$ grid. If $n \geq 5$, then it is possible to simultaneously extract Bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$, where $kn + 1 \leq a_1 < b_1 < b_2 < a_2 \leq (k + 1)n$ and $b_2 = b_1 + 1$, for some $0 \leq k \leq 1$.

Proof. Let $n \geq 5$. WLOG assume that $\{a_1, a_2, b_1, b_2\}$ all lie on the bottom row, so $k = 1$. Then G contains vertices such that $a_1 < b_1 < b_2 < a_2$ and $b_2 = b_1 + 1$. We can show that $G' < G$ where G' contains 2 line graphs, each with a desired bell pair.

To do this we first perform vertex deletions so that the bottom corner vertices are $\{a_1, a_2\}$. This is achieved by performing Z measurements on all vertices in columns left of a_1 and right of a_2 .

Next, we will ensure that $\{a_1, b_1\}$ and $\{b_2, a_2\}$ are neighbors. If they are already neighbors then move onto the next step. If one, or both of them are not, use the corner replacement protocol on a_1 and/or a_2 : $\rho_{a_1}^{b_1-1}$ and/or $\rho_{a_2}^{b_2+1}$. See definition 1.4.

Now we have a grid such that $\{a_1, a_2\}$ are the bottom corner vertices. Additionally, $b_1 = a_1 + 1$ and $b_2 = a_2 - 1$. Since $b_2 = b_1 + 1$, we know that the graph is now a 2×5 grid.

Thus in the 2×5 grid, $a_1 = 6$, $a_2 = 10$, $b_1 = 7$, and $b_2 = 9$.

We can now use the following protocol on the grid to achieve G' . First perform X measurements on vertex $\{4\}$ using vertex $\{5\}$ as a pivot, and then on vertex $\{2\}$ using vertex $\{1\}$ as a pivot. Then perform a local complementation on vertex $\{5\}$ and vertex $\{1\}$. Finally, perform a Z measurement on vertex $\{1\}$, vertex $\{8\}$, and then vertex $\{5\}$. Then $G' = Z_5 \circ Z_8 \circ Z_1 \circ \tau_1 \circ \tau_5 \circ X_2^1 \circ X_4^5 \circ \tau_8(G)$. See append G.

Thus G' is a graph containing the extracted bell pair $\{b_1, b_2\}$ and the line graph $\{a_1, 3, a_2\}$. Then by performing a Y measurement on vertex $\{3\}$ we can see that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

If $k=0$, you can use the same technique except performed on inverted vertices to show that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable. $\square$

Lemma 3.4. Let $G = (V, E)$ be a $2 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$, where $kn + 1 \leq a_1 < b_1 < b_2 < a_2 \leq (k + 1)n$ for some $0 \leq k \leq 1$.

Proof. It is easy to see that this follows directly from lemma 3.1, lemma 3.2, and lemma 3.3. $\square$

Lemma 3.5. Let $G = (V, E)$ be a $2 \times n$ grid. If $n \geq 4$, then it is possible to simultaneously extract Bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$, where $kn + 1 \leq a_1 < a_2 < b_1 < b_2 \leq (k + 1)n$ and $b_1 = a_2 + 1$, for some $0 \leq k \leq 1$.

Proof. Let $n \geq 4$. WLOG assume that $\{a_1, a_2, b_1, b_2\}$ all lie on the bottom row, so $k = 1$. Then G contains vertices such that $a_1 < a_2 < b_1 < b_2$ and $b_1 = a_2 + 1$. We can show that $G' < G$ where G' is a 2×4 grid s.t $a_1 = 5$, $a_2 = 6$, $b_1 = 7$, and $b_2 = 8$.

To do this we first perform vertex deletions so that the bottom corner vertices are $\{a_1, b_2\}$. This is achieved by performing Z measurements on all vertices in columns left of a_1 and right of b_2 .

Next, we will ensure that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are neighbors. If they are already neighbors then move onto the next step. If one, or both of them are not, use the corner replacement protocol on a_1 and/or b_2 : $\rho_{a_1}^{a_2-1}$ and/or $\rho_{b_2}^{b_1+1}$. See definition 1.4.

Now we have a grid such that $\{a_1, b_2\}$ are the bottom corner vertices. Additionally, $a_1 = a_2 - 1$ and $b_2 = b_1 + 1$. Since $b_1 = a_2 + 1$, we know that the graph is now the 2×4 grid.

Thus in the 2×4 grid, $a_1 = 5$, $a_2 = 6$, $b_1 = 7$, and $b_2 = 8$.

We can now use the following protocol on G' to simultaneously extract $\{a_1, a_2\}$ and $\{b_1, b_2\}$. First perform Y measurements on vertex $\{2\}$, and then on vertex $\{3\}$. Finally, perform a Z measurement on vertex $\{1\}$ and then vertex $\{4\}$. Then $G'' = Z_4 \circ Z_1 \circ Y_3 \circ Y_2(G')$. See appendix H.

Thus G'' contains the extracted bell pairs. So, we can see that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable.

If $k=0$, you can use the same technique except performed on inverted vertices to show that $\{a_1, a_2\}$ and $\{b_1, b_2\}$ are simultaneously extractable. $\square$

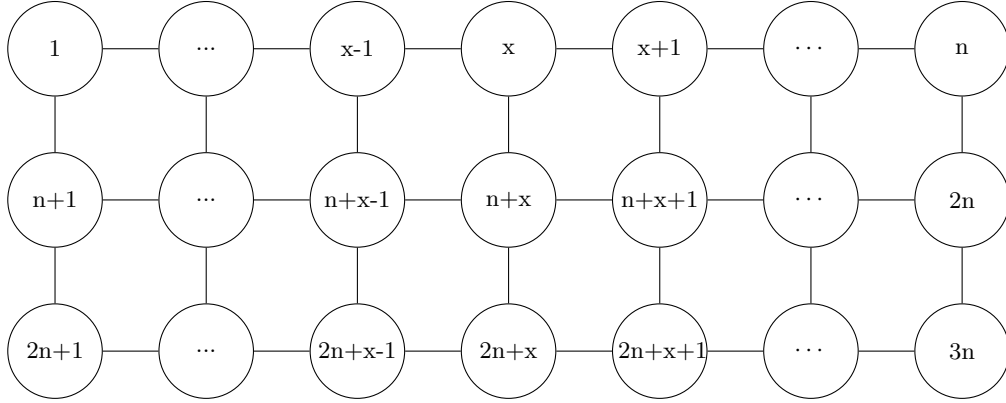
It is not hard to see that since all these cases work for a $2 \times n$ grid, one could easily extend them to a $m \times n$ grid.

References

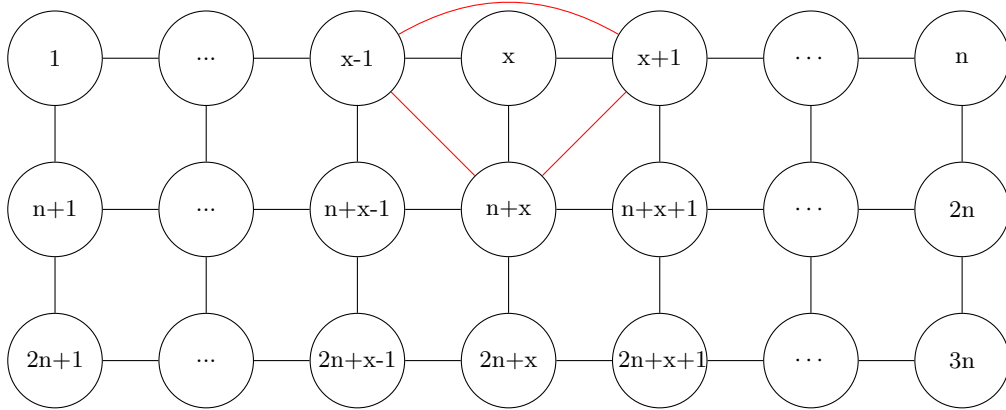
- [1] Vaisakh Mannalath and Anirban Pathak. Entanglement routing and bottlenecks in grid networks. *Annalen der Physik*, 537(5):2400316, 2025.
- [2] Derek Zhang. Bell pair extraction using graph foliage techniques. *Journal of Mathematical Physics*, 66(2):022204, 02 2025.

Appendices

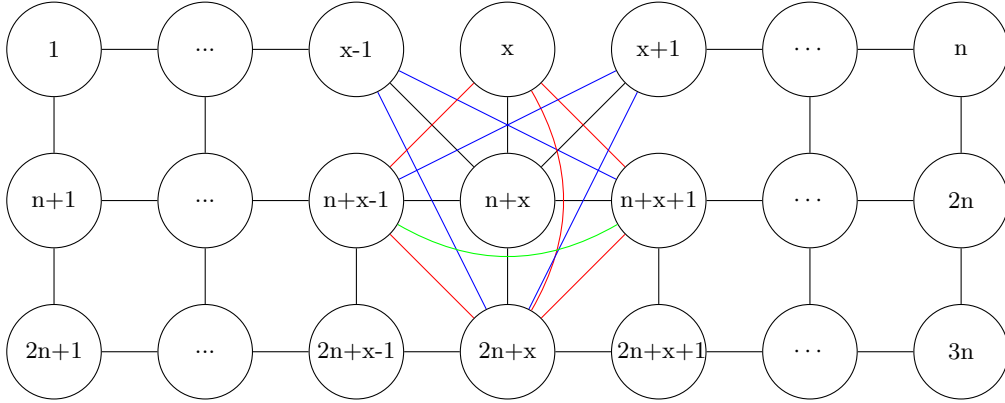
A Intermediary graphs for definition 1.1



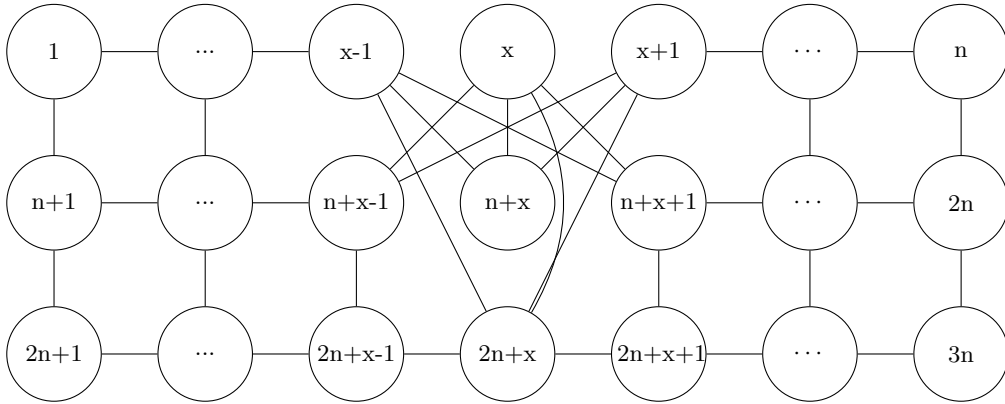
(a) G is a $3 \times n$ grid where $n \geq 3$. We will perform the crossing protocol on G to get $G_5 = \chi_{n+x}(G)$.



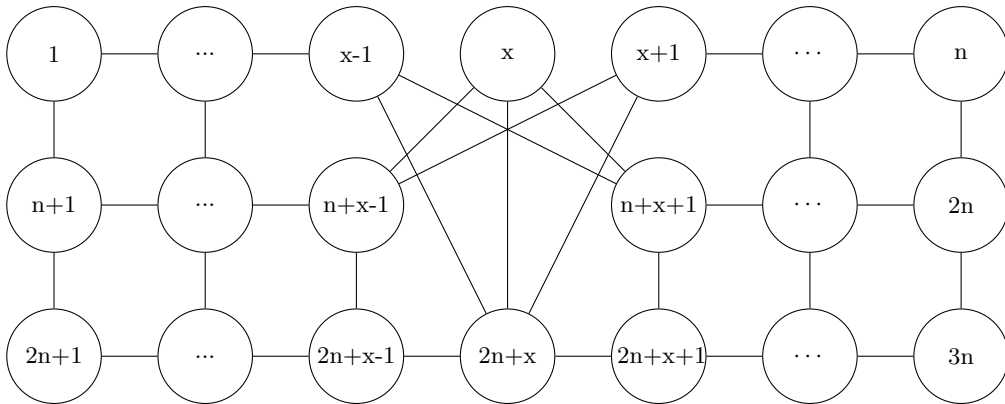
(b) G_1 , the first intermediary graph. Perform a local complementation on vertex x to get $G_1 = \tau_x(G)$. Added edges from the LC are colored.



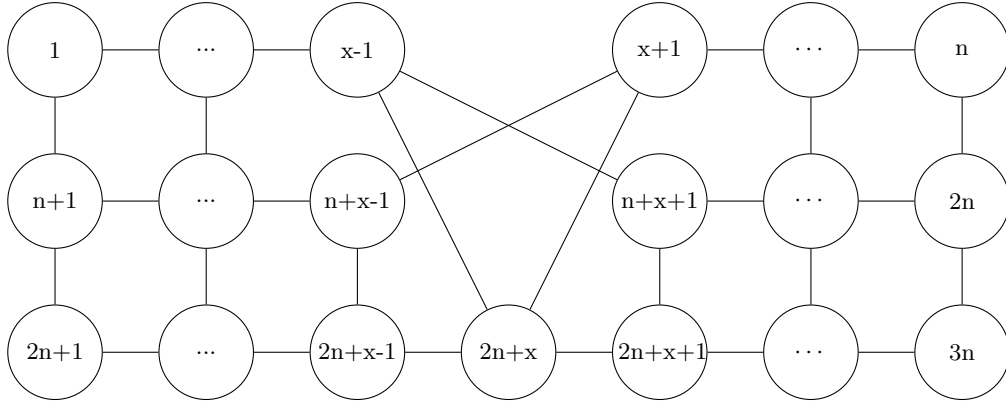
(c) G_2 , the second intermediary graph. Perform a local complementation on vertex $n+x$ to get $G_2 = \tau_{n+x}(G_1)$. Added edges from the LC are colored.



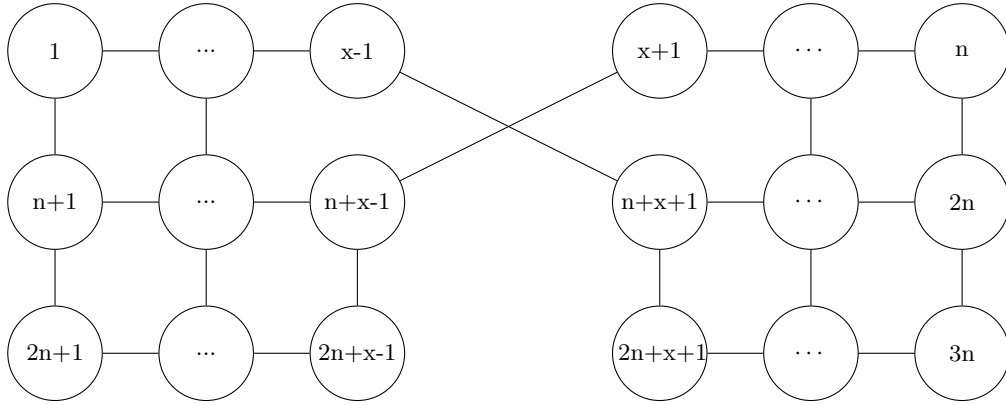
(d) G_3 , the third intermediary graph. Perform a local complementation on vertex x to get $G_3 = \tau_x(G_2)$. Added edges from the LC are colored.



(e) G_4 , the fourth intermediary graph. Perform a vertex deletion on vertex $n+x$ to get $G_4 = G_3 \setminus V_{n+x}$.



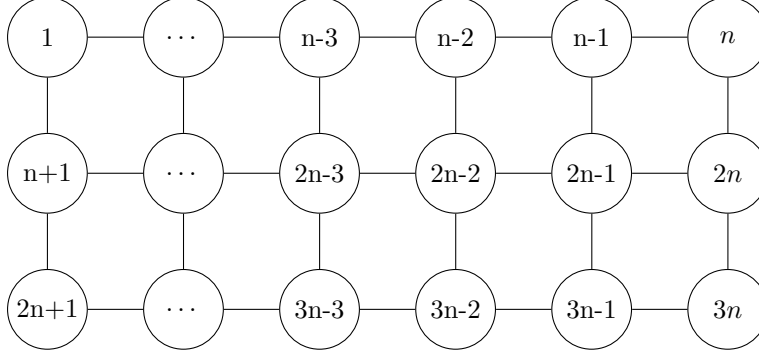
(f) G_5 , the fifth intermediary graph. Perform a vertex deletion on vertex x to get $G_5 = G_4 \setminus V_x$.



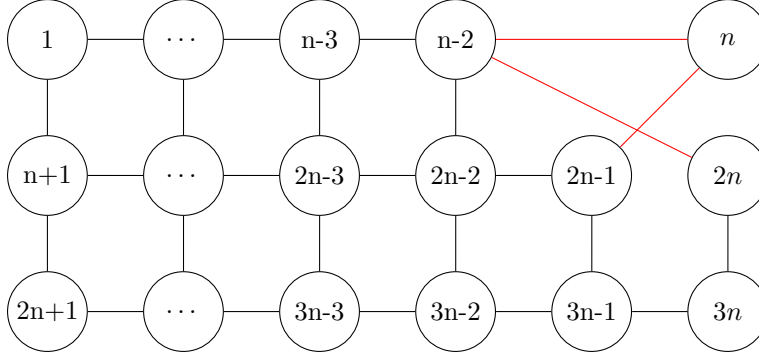
(g) G' , the resulting graph. Perform a vertex deletion on vertex $2n+x$ to get $G' = G_5 \setminus V_{2n+x} = \chi_{n+x}(G)$.

Figure 12

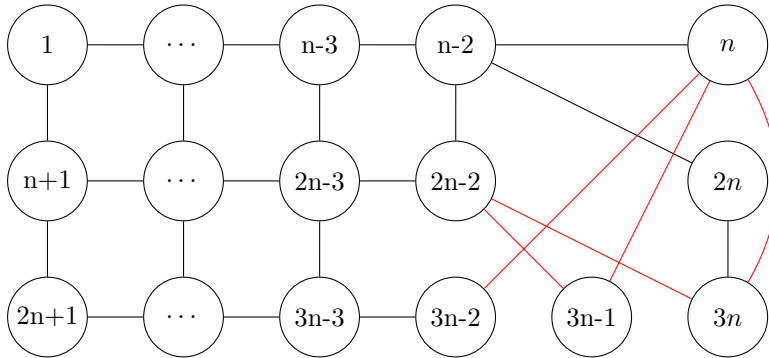
B Intermediary graphs for definition 1.2



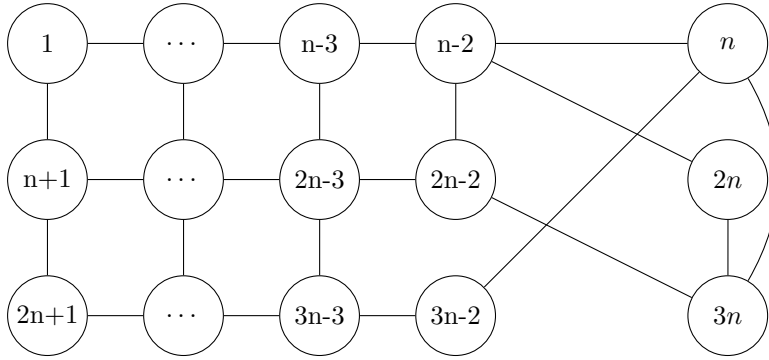
(a) G is a $3 \times n$ grid where $n \geq 4$. We will perform the flipping protocol on G to get $G' = \varphi(G)$.



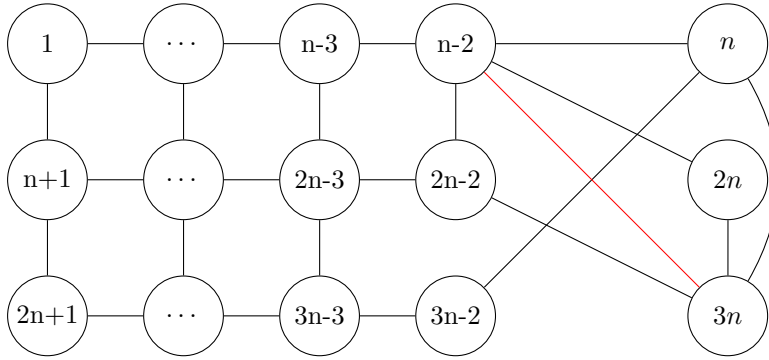
(b) G_1 , the first intermediary graph. Perform a X measurement on vertex $\{n-1\}$, using $\{n\}$ as a pivot, to get $G_1 = X_{n-1}^n(G)$. Added edges from the LCs are colored.



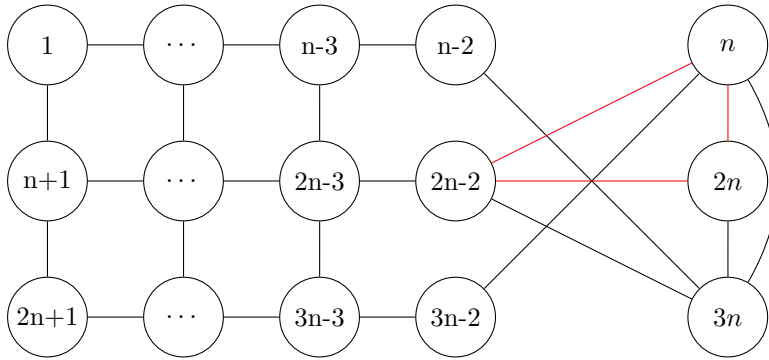
(c) G_2 , the second intermediary graph. Perform a X measurement on vertex $\{2n-1\}$, using $\{3n-1\}$ as a pivot, to get $G_2 = X_{2n-1}^{3n-1}(G_1)$. Added edges from the LCs are colored.



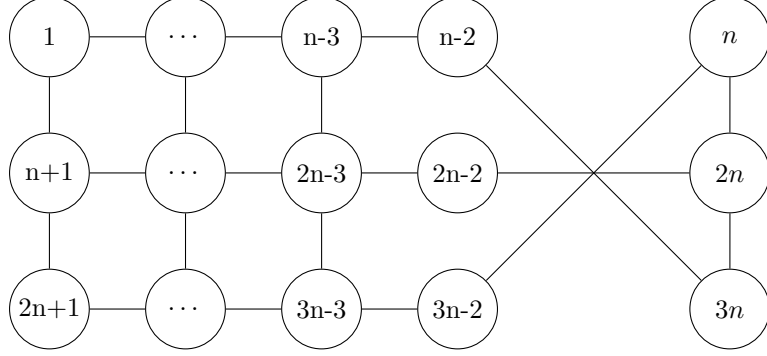
(d) G_3 , the third intermediary graph. Perform a Z measurement on vertex $\{3n-1\}$ to get $G_3 = Z_{3n-1}(G_2)$.



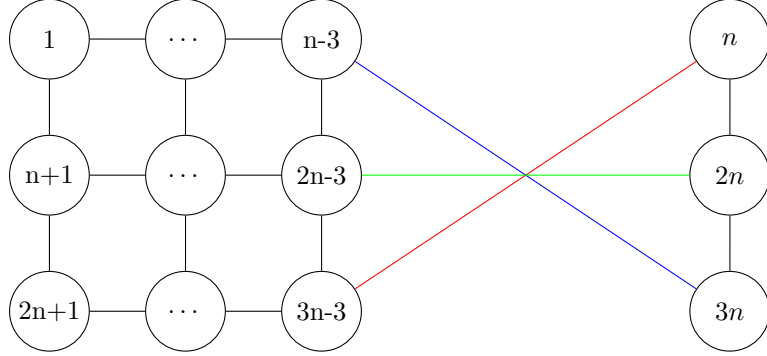
(e) G_4 , the fourth intermediary graph. Perform a local complementation on vertex $\{2n\}$ to get $G_4 = \tau_{2n}(G_3)$. Added edges from the LCs are colored.



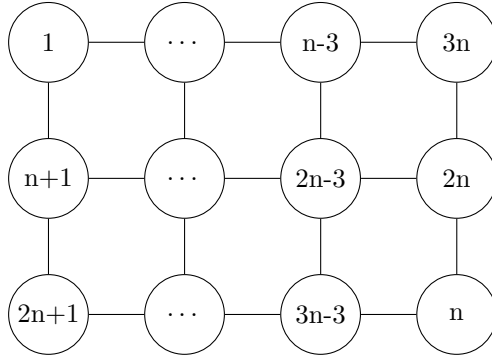
(f) G_5 , the fifth intermediary graph. Perform a local complementation on vertex $\{3n\}$ to get $G_5 = \tau_{3n}(G_4)$. Added edges from the LCs are colored.



(g) G_6 , the sixth intermediary graph. Perform a local complementation on vertex $\{2n\}$ to get $G_6 = \tau_{2n}(G_5)$.



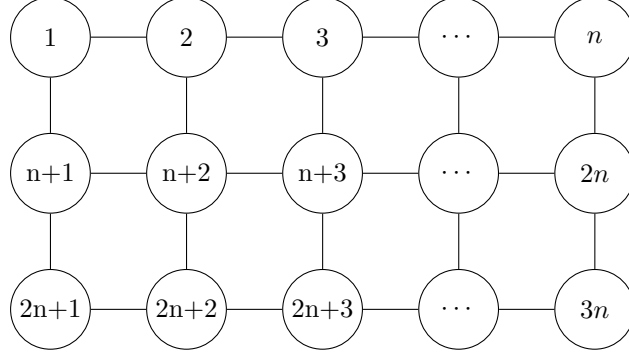
(h) G' , the resulting graph. Perform Y measurements on vertices $\{n-2, 2n-2, 3n-2\}$ to get $G_6 = Y_{3n-2} \circ Y_{2n-2} \circ Y_{n-2}(G_5)$.



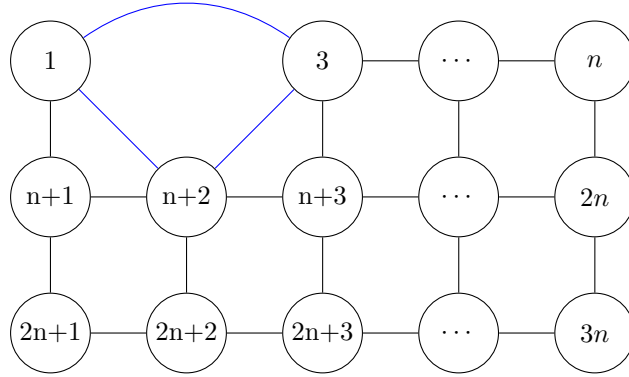
(i) G' , the resulting graph. $G' = \varphi(G)$

Figure 13

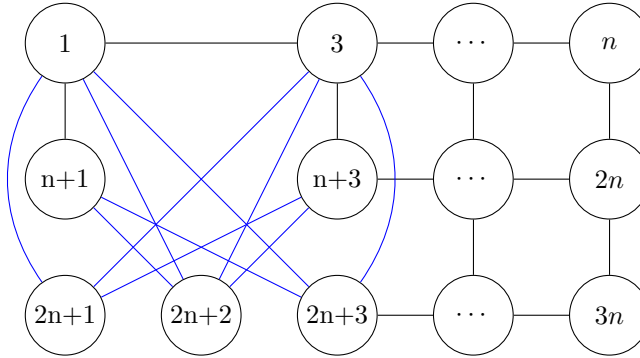
C Intermediary graphs for definition 1.3



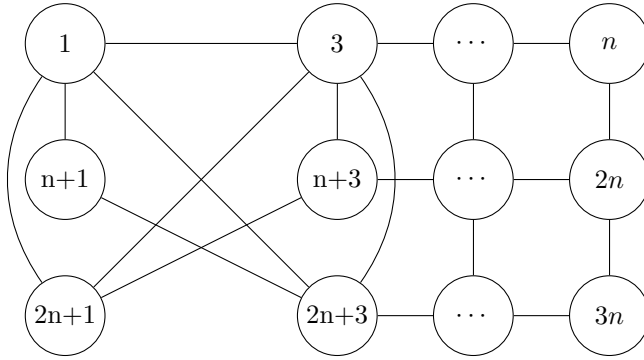
(a) G is a $3 \times n$ grid where $n \geq 3$. We will perform the Corner Extraction protocol on G to get $G' = \kappa(G)$.



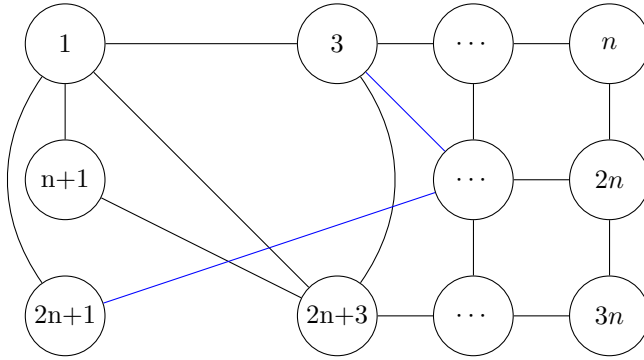
(b) G_1 , the first intermediary graph. Perform a Y measurement on vertex $\{2\}$ to get $G_1 = Y_2(G)$. Added edges from the LCs are colored.



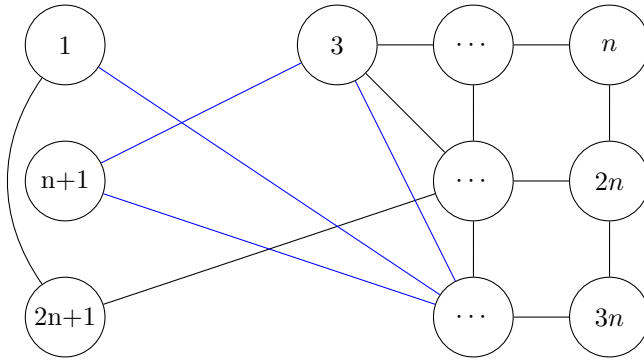
(c) G_2 , the second intermediary graph. Perform a X measurement on vertex $\{n+2\}$, using $\{2n+2\}$ as a pivot, to get $G_2 = X_{n+2}^{2n+2}(G_1)$. Added edges from the LCs are colored.



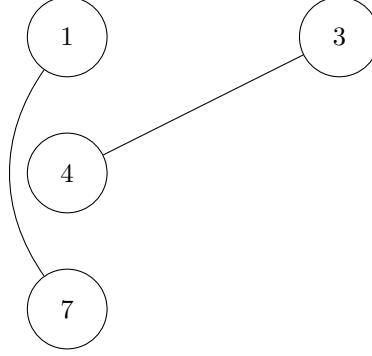
(d) G_3 , the third intermediary graph. Perform a Z measurement on vertex $\{2n+2\}$ to get $G_3 = Z_{2n+2}(G_2)$.



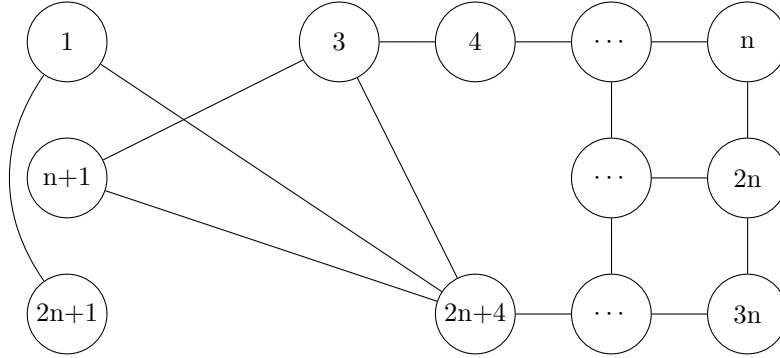
(e) G_4 , the fourth intermediary graph. Perform a Y measurement on vertex $\{n+3\}$ to get $G_4 = Y_{n+3}(G_3)$. Added edges from the LCs are colored.



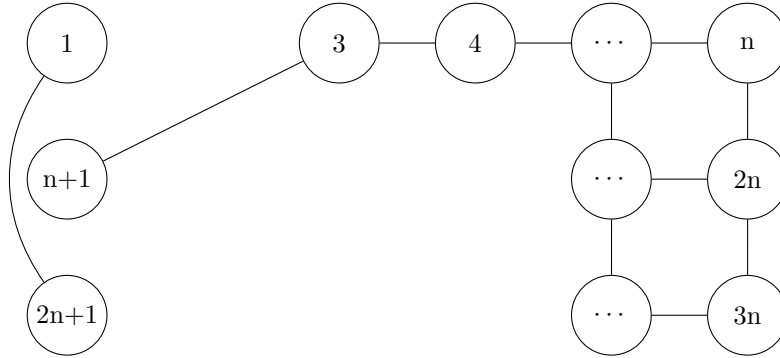
(f) G_5 , the fifth intermediary graph. Perform a Y measurement on vertex $\{2n+3\}$ to get $G_5 = Y_{2n+3}(G_4)$. Added edges from the LCs are colored.



(g) If $n = 3$, $G' = G_5 = \kappa(G)$. G' the resulting graph.



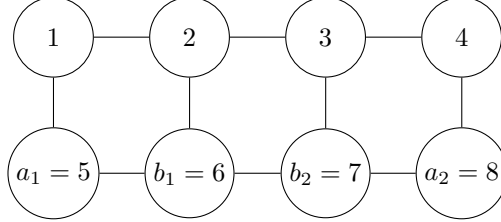
(h) If $n \geq 4$, G_6 , the sixth intermediary graph. Perform a Z measurement on vertex $\{n + 4\}$ to get $G_6 = Z_{n+4}(G_5)$.



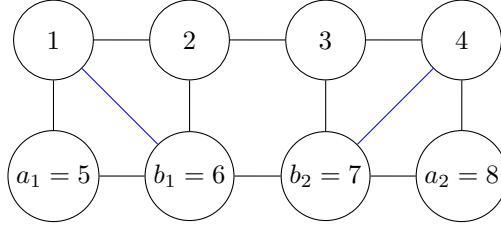
(i) If $n \geq 4$, G' , the resulting graph. $G' = \kappa(G)$. Perform a Z measurement on vertex $\{2n + 4\}$ to get $G' = G_7 = Z_{2n+4}(G_6)$.

Figure 14

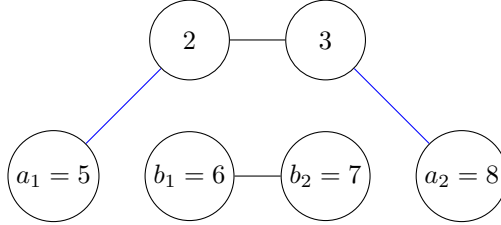
D Intermediary Graphs for Lemma 3.1



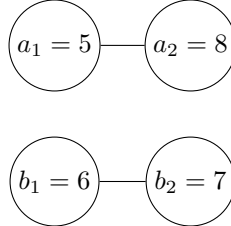
(a) Base Case: G is a 2×4 grid where $5 \leq a_1 < b_1 < b_2 < a_2 \leq 8$. We will simultaneously extract $\{a_1, a_2\}$ $\{b_1, b_2\}$.



(b) Base Case: G_1 , the first intermediary graph. Perform local complementations on vertices $\{5, 8\}$ to get $G_1 = \tau_5 \circ \tau_8(G)$. Added edges from the LCs are colored.

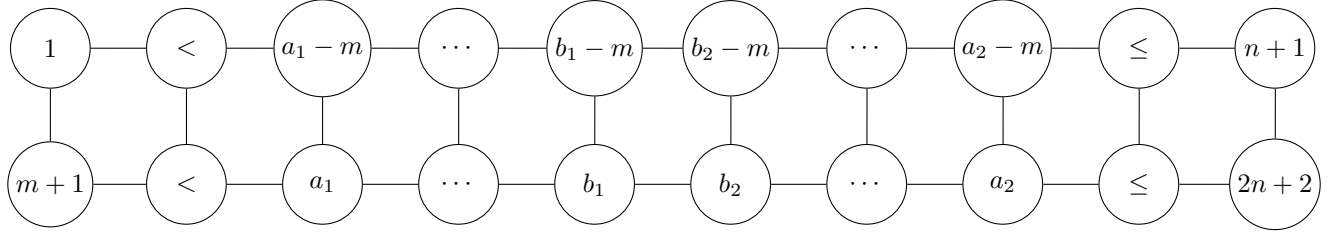


(c) Base Case: G_2 , the second intermediary graph. Perform Y measurements on vertices $\{1, 4\}$ to get $G_2 = Y_1 \circ Y_4(G_1)$. Added edges from the LCs are colored.

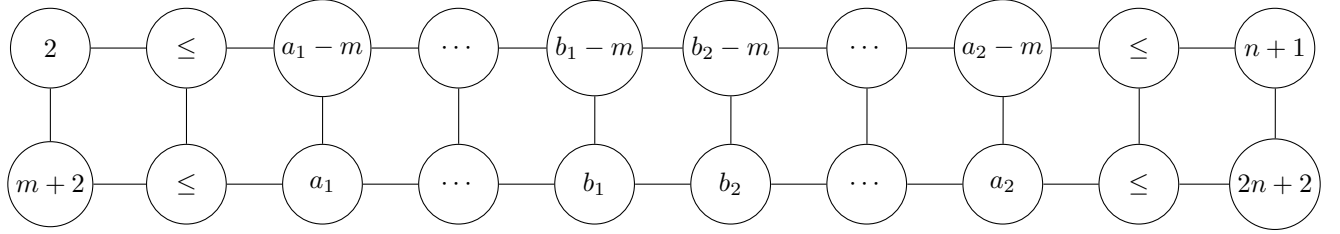


(d) Base Case: G' , the resulting graph containing the extracted bell pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$. Perform Y measurements on vertices $\{2, 3\}$ to get $G' = Y_2 \circ Y_3(G_2)$.

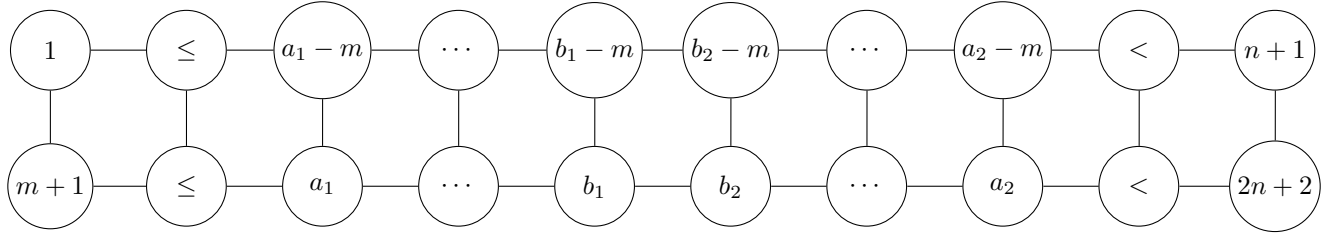
Figure 15



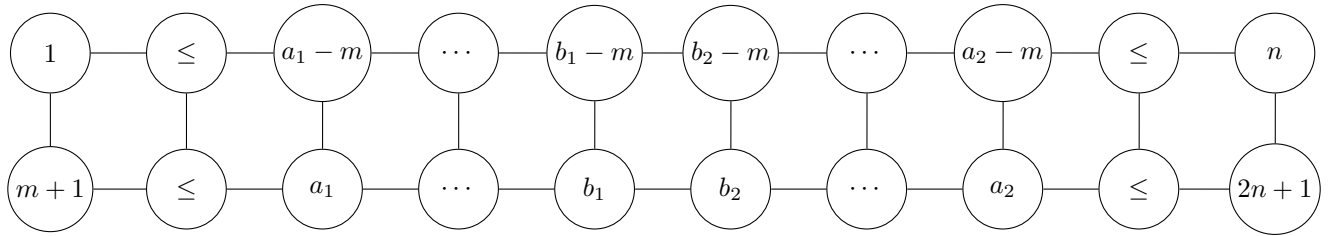
(a) Case 1: G is a $2 \times (n + 1)$ grid where $(n + 1) + 1 < a_1 < b_1 < b_2 < a_2 \leq 2(n + 1)$. Note: $m = n + 1$.



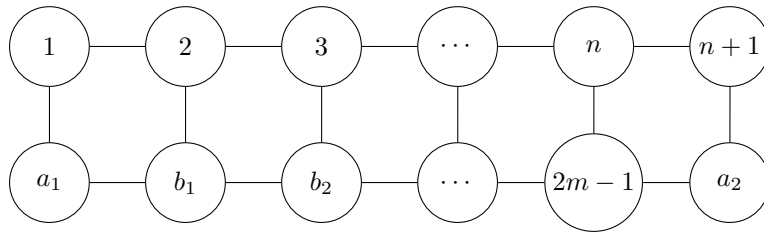
(b) Case 1: We can safely delete the first column and preserve all vertices of the bell pairs. The resulting grid is a $2 \times n$ grid, where $(n + 1) + 2 \leq a_1 < b_1 < b_2 < a_2 \leq 2(n + 1)$



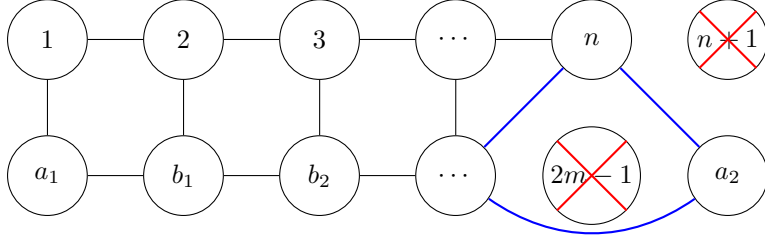
(c) Case 2: G is a $2 \times (n + 1)$ grid where $(n + 1) + 1 \leq a_1 < b_1 < b_2 < a_2 < 2(n + 1)$. Note: $m = n + 1$



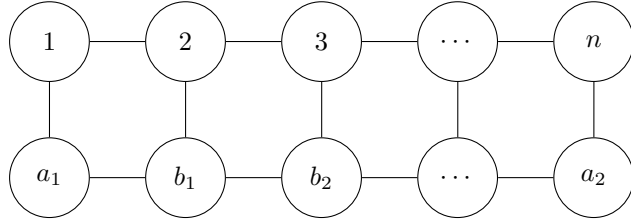
(d) Case 2: We can safely delete the last column and preserve all vertices of the bell pairs. The resulting grid is a $2 \times n$ grid, where $(n + 1) + 1 \leq a_1 < b_1 < b_2 < a_2 \leq 2n + 1$



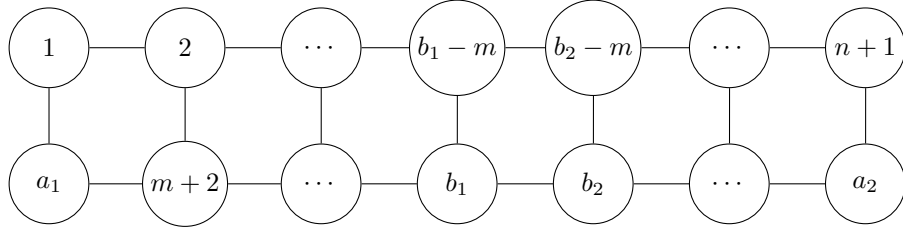
(e) Case 3: G is a $2 \times (n + 1)$ grid. $a_1 = m + 1$, $a_2 = 2m$ and $b_1 = a_1 + 1$. Then we know $b_2 = a_1 + 2$ and $b_2 \neq 2m - 1$. Note: $m = n + 1$.



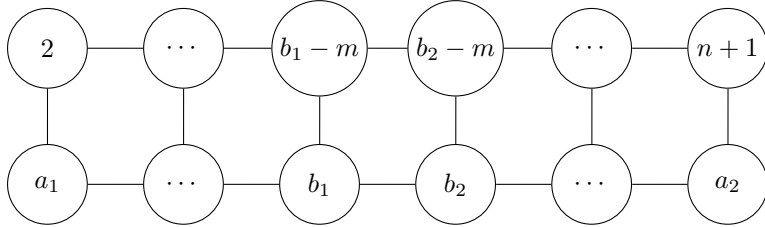
(f) Case 3: Delete the $\{n+1\}$ vertex and perform a Y measurement on $\{2m-1\}$. $G_1 = Y_{2m-1} \circ Z_{n+1}(G)$ Added edges from the LCs are colored.



(g) Case 3: We perform a local complementation on vertex $\{a_2\}$, so $G' = \tau_{a_2}(G_1)$. The resulting grid is a $2 \times n$ grid that preserves all vertices of the bell pairs.



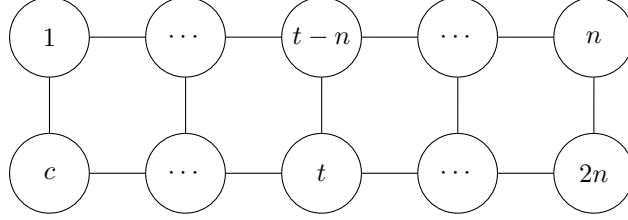
(h) Case 4: G is a $2 \times (n+1)$ grid. $a_1 = m+1$, $a_2 = 2m$ and $b_1 > a_1 + 1$. Then we know a_1 and b_1 cannot be neighbors. Note: $m = n+1$.



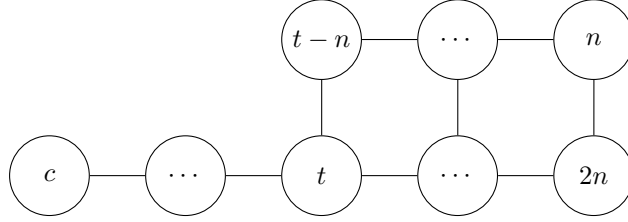
(i) Case 4: We perform a similar technique as in case 3, so $G' = \tau_{a_1} \circ Y_{m+2} \circ Z_1(G_1)$. The resulting grid is a $2 \times n$ grid that preserves all vertices of the bell pairs.

Figure 16

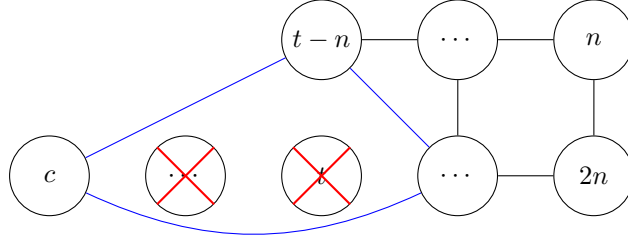
E Intermediary graphs for definition 1.4



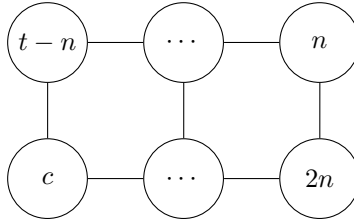
(a) G is a $2 \times n$ grid where $n \geq 3$. We will perform the Corner Replacement Protocol, ρ_c^t , to get $G' = \rho_c^t(G)$. For simplicity we will just show the case where $c = n + 1$.



(b) G_1 , the first intermediary graph. Perform Z measurements on all vertices between $(1, t-n)$, as well as vertex $\{1\}$, to get $G_1 = Z_{t-n-1} \circ \dots \circ Z_2 \circ Z_1(G)$.



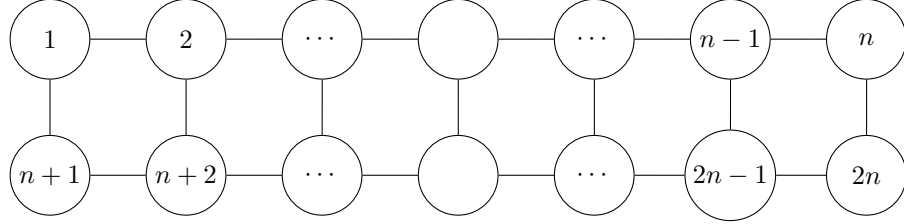
(c) G_2 , the second intermediary graph. Perform Y measurements on all vertices between $[c+1, t]$, inclusive, to get $G_2 = Y_t \circ \dots \circ Y_{c+2} \circ Y_{c+1}(G)$. Added edges from the LC's are colored.



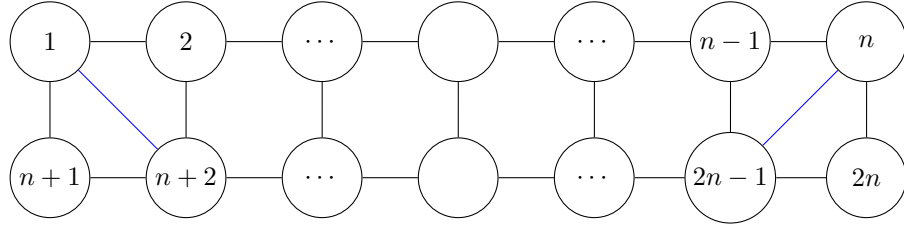
(d) $G' = \rho_c^t(G)$, the resulting graph. Perform a local complementation on vertex $\{c\}$ to get $G' = G_3 = \tau_c(G_2)$.

Figure 17

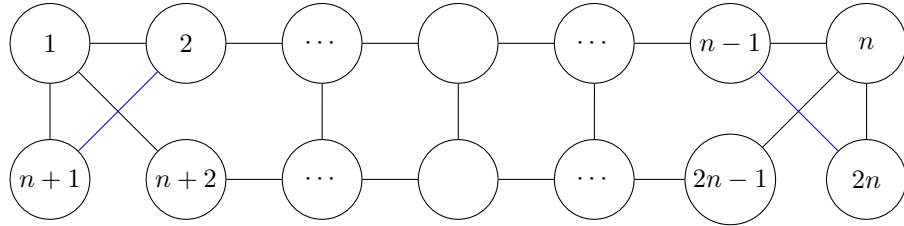
F Intermediary graphs for definition 1.5



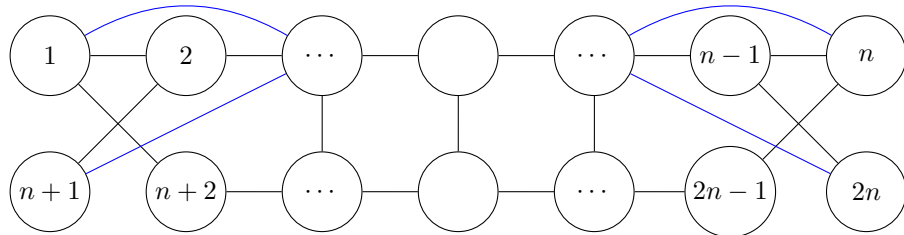
(a) G is a $2 \times n$ grid where $n \geq 5$. We will perform the Corner Folding Protocol, ξ , to get $G' = \xi(G)$.



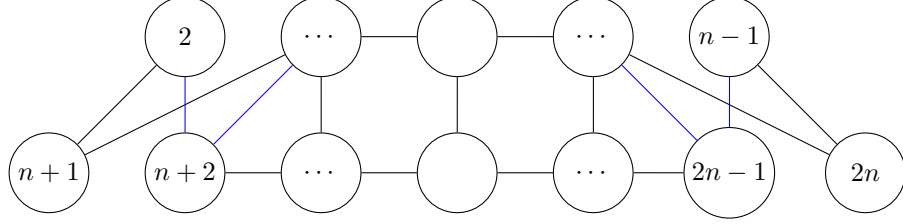
(b) G_1 , the first intermediary graph. Perform local complementations on vertices $\{n+1, 2n\}$, to get $G_1 = \tau_{2n} \circ \tau_{n+1}(G)$. Added edges from the LC's are colored.



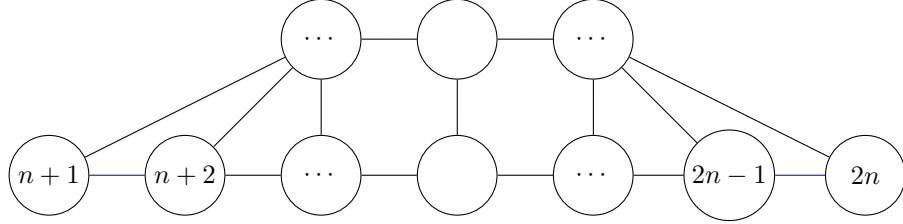
(c) G_2 , the second intermediary graph. Perform local complementations on vertices $\{1, n\}$, to get $G_2 = \tau_n \circ \tau_1(G_1)$. Added edges from the LC's are colored.



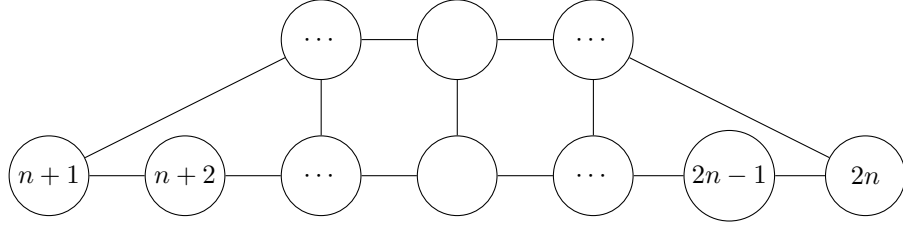
(d) G_3 , the third intermediary graph. Perform local complementations on vertices $\{2, n-1\}$, to get $G_3 = \tau_{n-1} \circ \tau_2(G_2)$. Added edges from the LC's are colored.



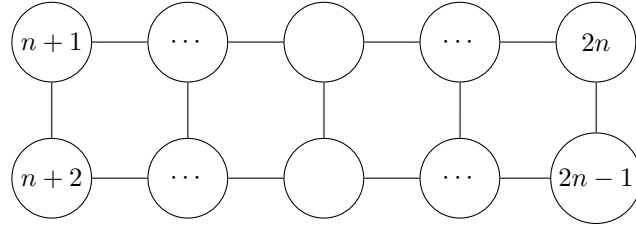
(e) G_4 , the fourth intermediary graph. Perform Y measurement on vertices $\{1, n\}$, to get $G_4 = Y_n \circ Y_1(G_3)$. Added edges from the LC's are colored.



(f) G_5 , the fifth intermediary graph. Perform Y measurement on vertices $\{2, n-1\}$, to get $G_5 = Y_{n-1} \circ Y_2(G_4)$. Added edges from the LC's are colored.



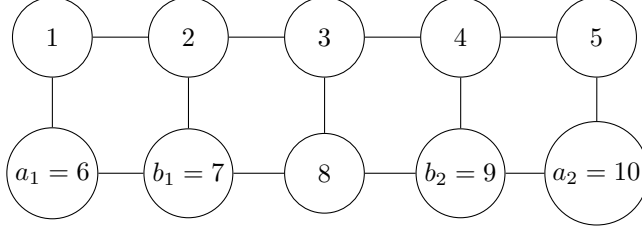
(g) G' , the resulting graph. Perform local complementations on vertices $\{n+1, 2n\}$, to get $G' = G_6 = \tau_{2n} \circ \tau_{n+1}(G_5)$. Added edges from the LC's are colored.



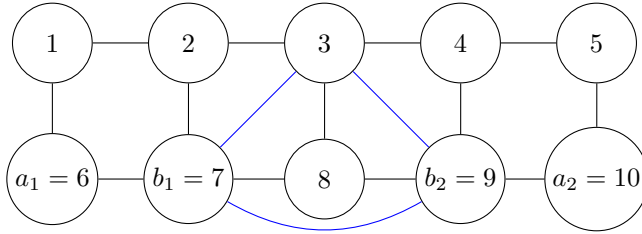
(h) G' is a $2 \times (n-2)$ grid with corner vertices $V_c \in \{n+1, n+2, 2n, 2n-1\}$.

Figure 18

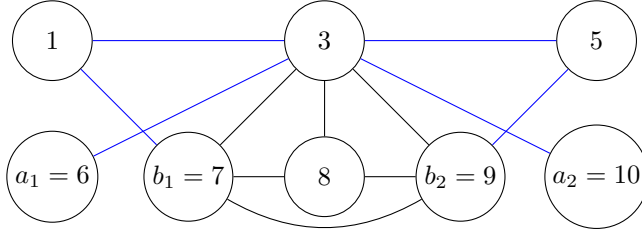
G Intermediary Graphs for Lemma 3.3



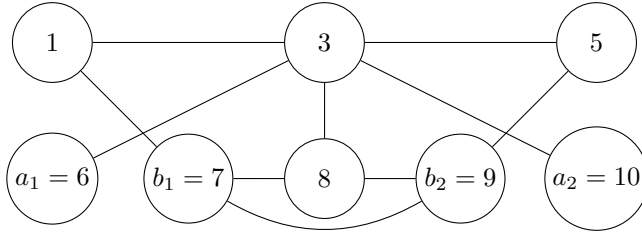
(a) G is a 2×5 grid. We will simultaneously extract $\{a_1, a_2\}$ $\{b_1, b_2\}$.



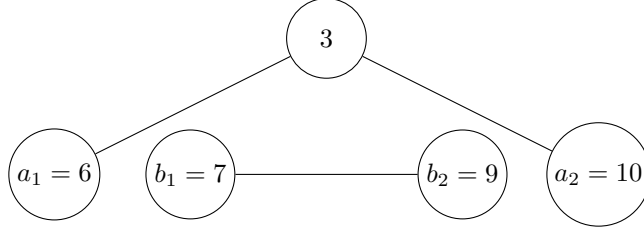
(b) G_1 , the first intermediary graph. Perform a local complementation on vertex $\{8\}$. $G_1 = \tau_8(G)$. Added edges from the LCs are colored.



(c) G_2 , the second intermediary graph. Perform X measurements on vertex $\{4\}$ using vertex $\{5\}$ as a pivot, and then on vertex $\{2\}$ using vertex $\{1\}$ as a pivot. $G_2 = X_2^1 \circ X_4^5(G_1)$. Added edges from the LCs are colored.



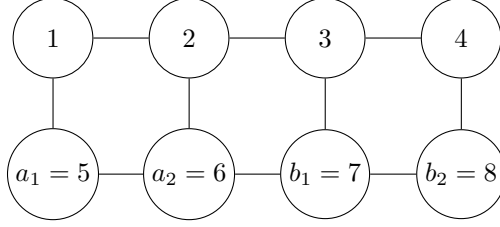
(d) G_3 , the third intermediary graph. Perform a local complementation on vertex $\{5\}$ and then on vertex $\{1\}$. $G_3 = \tau_1 \circ \tau_5(G_2)$.



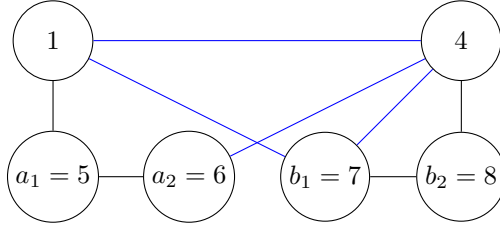
(e) G' , the resulting graph. Perform a Z measurement on vertex $\{1\}$, vertex $\{8\}$, and then on vertex $\{5\}$. $G' = Z_5 \circ Z_8 \circ Z_1(G_3) = Z_5 \circ Z_8 \circ Z_1 \circ \tau_1 \circ \tau_5 \circ X_2^1 \circ X_4^5 \circ \tau_8(G)$.

Figure 19

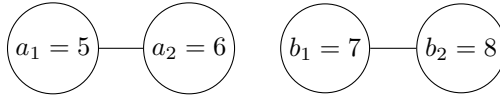
H Intermediary Graphs for Lemma 3.5



(a) G' is a 2×4 grid. We will simultaneously extract $\{a_1, a_2\}$ $\{b_1, b_2\}$.



(b) G_1 , the first intermediary graph. Perform a Y measurement on vertex $\{2\}$ and then on vertex $\{3\}$. $G_1 = Y_3 \circ Y_2(G')$. Added edges from the LCs are colored.



(c) G'' , the resulting graph. Perform a Z measurement on vertex $\{1\}$ and then vertex $\{4\}$. $G'' = Z_4 \circ Z_1(G_1) = Z_4 \circ Z_1 \circ Y_3 \circ Y_2(G')$.

Figure 20

I Code

```
In [1]: import scipy
        #A python library that is helpful when dealing with graphs
        import networkx as nx
        import matplotlib.pyplot as plt

In [2]: ## some useful functions

        ##draw the grid graph, can choose to lock the vertices into grid structure
        def draw_graph( graph, position = False, plot_size = (5,5)):

            plt.figure(figsize= plot_size)

            if position == False:

                nx.draw(
                    graph,
                    with_labels=True,
                    node_color="skyblue",
                    node_size=600,
                    font_size=8,
                    font_weight = 'bold',
                    edge_color="gray",)

            else:
                nx.draw(
                    graph,
                    pos= position,
                    with_labels=True,
                    node_color="skyblue",
                    node_size=600,
                    font_size=8,
                    font_weight = 'bold',
                    edge_color="gray",)

            plt.show()

        #function to determine where each node needs to be for gridlock for a grid
        #defined by integers
        def convert_position_to_integers (integer_Graph, cols ):
            pos_int = {}
            for node in integer_Graph.nodes():
                node_mod = node- 1
                col_idx = node_mod % cols
                row_idx = node_mod // cols
                pos_int[node] = (col_idx, -row_idx)
            return pos_int

In [3]: ##perform local complementation on a node using a graph defined by integers
        def local_complementation_integers (vertex, Graph, show_graph = False,
                                              gridlock = False, plot_size = (3,3),
                                              position_int = None):

            # copy the graph
            G = Graph.copy()
```

```

#extract neighbor subgraph of vertex
neighbors = list(G.neighbors(vertex))
G_sub = G.subgraph(neighbors)

#complement the subgraph
G_sub_complement = nx.complement(G_sub)

#delete existing edges between neighbors
G.remove_edges_from(G_sub.edges())

#add complement edges between neighbors
G.add_edges_from(G_sub_complement.edges())

#plot graph
if show_graph== True:
    if gridlock == False:
        draw_graph(G, plot_size = plot_size)
    else:
        draw_graph(G, position_int, plot_size = plot_size)

return G

```

In [4]: *#Functions for performing Graph Operations/Pauli Measurements*
##Each function has the option to draw the resulting graph after performing
#the operation

```

def Z_graph_operation(vertex_to_delete, graph_int, show_graph = False,
                      gridlock = False, plot_size = (3,3),
                      position_int = None):

    G = graph_int.copy()

    #delete vertex
    G.remove_node(vertex_to_delete)

    #plot graph
    if show_graph== True:
        if gridlock == False:
            draw_graph(G, plot_size = plot_size)
        else:
            draw_graph(G, position_int, plot_size = plot_size)

    return G

def Y_graph_operation(vertex_to_delete, graph_int, show_graph = False,
                      gridlock = False, plot_size = (3,3),
                      position_int = None):

    G = graph_int.copy()

    #perform LC on vertex
    G = local_complementation_integers(vertex_to_delete, G)

    #delete vertex
    G.remove_node(vertex_to_delete)

```

```

    #plot graph
    if show_graph== True:
        if gridlock == False:
            draw_graph(G, plot_size = plot_size)
        else:
            draw_graph(G, position_int, plot_size = plot_size)

    return G

def X_graph_operation(vertex_to_delete, graph_int, pivot = False,
                      show_graph = False, gridlock = False,
                      plot_size = (3,3), position_int = None):

    neighbors = list(graph_int.neighbors(vertex_to_delete))

    ##check to see if the vertex has any neighbors
    if not neighbors:
        print ("vertex has no neighbors, unable to perform operation")
        return

    #pick a pivot if unspecified
    if pivot == False:
        pivot = neighbors[0]

    #ensure the pivot choice is a neighbor to the desired vertex
    if pivot not in neighbors:
        print("pivot isnt a neighbor, unable to perform operation")
        return graph_int

    G = graph_int.copy()

    #perform LC's on vertex and pivot
    G = local_complementation_integers(pivot, G)
    G = local_complementation_integers(vertex_to_delete, G)
    G = local_complementation_integers(pivot, G)

    #delete vertex
    G.remove_node(vertex_to_delete)

    #plot graph
    if show_graph== True:
        if gridlock == False:
            draw_graph(G, plot_size = plot_size)
        else:
            draw_graph(G, position_int, plot_size = plot_size)

    return G

```

In [1]: *#functions for protocols on 3xn and 2xn grids*

```

#for a 3xn graph
def crossing_protocol(vertex_a, graph_int, cols, pivot = False,
                     show_graph = False, gridlock = False,
                     plot_size = (3,3), position_int = None):

```



```

neighbors = list(graph_int.neighbors(vertex_a))

#check to see if vertex a is a valid choice
required_neighbors = [vertex_a - cols, vertex_a - 1,
                      vertex_a + 1, vertex_a + cols]
for neighbor in required_neighbors:
    if neighbor not in neighbors:
        print ("a is not a valid option, pick another vertex.")
        return

G = graph_int.copy()

#perform X measurement on vertex a using a-n as the pivot
G = X_graph_operation(vertex_a, graph_int, pivot = vertex_a-cols,
                      show_graph= True, gridlock = True,
                      plot_size= (10,3), position_int= pos_int)

#perform vertex deletions
G = Z_graph_operation( vertex_a-cols,G )
G = Z_graph_operation( vertex_a+cols,G )

#plot graph
if show_graph== True:
    if gridlock == False:
        draw_graph(G, plot_size = plot_size)
    else:
        draw_graph(G, position_int, plot_size = plot_size)

return G

# for a 3xn graph
def flipping_protocol(graph_int, cols, show_graph = False, gridlock = False,
                     plot_size = (3,3), position_int = None):

    #check to see if grid is sufficiently large
    if cols < 4:
        print ("Grid is not large enough.")
        return

    G = graph_int.copy()

    #perform X and Z measurements
    G = X_graph_operation( cols -1, G, pivot= cols )
    G = X_graph_operation( 2*cols -1, G, pivot= 3*cols-1 )
    G = Z_graph_operation( 3*cols-1 ,G )

    #perform LC's
    G = local_complementation_integers( 2*cols ,G )
    G = local_complementation_integers( 3*cols ,G )
    G = local_complementation_integers( 2*cols ,G )

    #perform Y measurements
    G = Y_graph_operation(cols-2 ,G )
    G = Y_graph_operation(2*cols-2 ,G )

```

```

G = Y_graph_operation(3*cols-2 ,G )

#plot graph
if show_graph== True:
    if gridlock == False:
        draw_graph(G, plot_size = plot_size)
    else:
        draw_graph(G, position_int, plot_size = plot_size)

return G

# for a 3xn graph
def extract_corners(graph_int, cols, show_graph = False, gridlock = False,
                    plot_size = (3,3), position_int = None,
                    invert = False):

    #check to see if grid is sufficiently large
    if cols < 3:
        print ("Grid is not large enough.")
        return

    G = graph_int.copy()

    #perform X Y Z measurements for all n
    if invert == True:
        G = Y_graph_operation( 2*cols+2, G)
        G = X_graph_operation( cols +2, G, pivot= 2 )
        G = Z_graph_operation( 2 ,G )
        G = Y_graph_operation( cols+3 ,G )
        G = Y_graph_operation( 3 ,G )
    else:
        G = Y_graph_operation( 2, G)
        G = X_graph_operation( cols +2, G, pivot= 2*cols+2 )
        G = Z_graph_operation( 2*cols+2 ,G )
        G = Y_graph_operation( cols+3 ,G )
        G = Y_graph_operation( 2*cols+3 ,G )

    #if n=3 the protocol is over
    if cols == 3:
        #plot graph
        if show_graph== True:
            if gridlock == False:
                draw_graph(G, plot_size = plot_size)
            else:
                draw_graph(G, position_int, plot_size = plot_size)

        return G

    #if n>=4 finish the protocol
    if invert == True:
        G = Z_graph_operation(cols+4 ,G )
        G = Z_graph_operation(4 ,G )
    else:
        G = Z_graph_operation(cols+4 ,G )
        G = Z_graph_operation(2*cols+4 ,G )

```

```

#plot graph
if show_graph== True:
    if gridlock == False:
        draw_graph(G, plot_size = plot_size)
    else:
        draw_graph(G, position_int, plot_size = plot_size)

return G

#for a 2xn graph
def corner_folding_protocol( graph_int, cols, show_graph = False,
                             gridlock = False, plot_size = (3,3),
                             position_int = None):

    #check to see if grid is sufficiently large
    if cols < 5:
        print("Grid is too small, please pick a 2xn grid with n>4")
        return

    G = graph_int.copy()

    #Perform LC's
    G = local_complementation_integers(cols+1, G)
    G = local_complementation_integers(2*cols, G)

    G = local_complementation_integers(1, G)
    G = local_complementation_integers(cols, G)

    G = local_complementation_integers(2, G)
    G = local_complementation_integers(cols-1, G)

    #Perform Y measurements
    G = Y_graph_operation(1, G)
    G = Y_graph_operation(2, G)
    G = Y_graph_operation(cols, G)
    G = Y_graph_operation(cols-1, G)

    #perform final LC's
    G = local_complementation_integers(cols+1, G)
    G = local_complementation_integers(2*cols, G)

    #plot graph
    if show_graph== True:
        if gridlock == False:
            draw_graph(G, plot_size = plot_size)
        else:
            draw_graph(G, position_int, plot_size = plot_size)

    return G

```

In [2]: *#perform the X Protocol on a grid of integers to extract a bell pair.*
#Must specify the desired ordered repeater line
def x_protocol_grid_integers(graph_int, ordered_repeater_line, bell_pair,
 show_graph = False, gridlock = False,
 plot_size = (3,3),

```

        position_int = None,
        show_X_measurement = False,
        gridlock_X = False,
        show_Z_measurement = False,
        gridlock_Z = False):

G = graph_int.copy()

#iteratively perform x measurements along the repeater line
for i in range (len(ordered_repeater_line)-2):
    #pivots are specified using the following: let V_i be the
    #ith vertex in the repeater line. Then if X(V_1), then
    #pivot = V_0; if x(v_{n-1}), then pivot = V_n; for any
    #intermeidary, use an adjacent pivot on the repeater line.

    if i == 0:
        p = ordered_repeater_line[0]
    elif i == len(ordered_repeater_line)-3:
        p = ordered_repeater_line[-1]
    else :
        neighbors = list(G.neighbors(ordered_repeater_line[i+1]))
        pivots = [v for v in neighbors if v in ordered_repeater_line]

        if not pivots:
            raise ValueError(f"No repeater neighbor found for vertex
                               {ordered_repeater_line[i+1]}")

        p = pivots[0]

G = X_graph_operation(ordered_repeater_line[i+1], G, pivot = p,
                      show_graph= show_X_measurement,
                      gridlock= gridlock_X, plot_size= plot_size)

#find all the neighbors of the bell pair
neighboring_vertices = list( G.neighbors(bell_pair[0]))
neighboring_vertices += list( G.neighbors(bell_pair[1]))
neighboring_vertices = [v for v in neighboring_vertices
                        if v not in bell_pair]

#perform z measurements on all neighbors
for v in neighboring_vertices:
    G = Z_graph_operation(v, G, show_graph = show_Z_measurement,
                          gridlock= gridlock_Z, plot_size= plot_size)

#plot graph
if show_graph== True:
    if gridlock == False:
        draw_graph(G, plot_size = plot_size)
    else:
        draw_graph(G, position_int, plot_size = plot_size)

return G

```

In []:

J Unfinished Lemmas

Lemma J.1. Let $G = (V, E)$ be a $m \times n$ grid. If $m, n \geq 3$, then it is possible to simultaneously extract all non-crossing, side bell pairs, $\{a_1, a_2\}$ and $\{b_1, b_2\}$

Proof.

□